

SI / RSI CUDA 并行与优化逻辑说明

当前 Figure 5 实现：S4 coarse SI，S128 fine SI / RSI / RSI-tail，8192 样本

对象	当前说明
空间网格	30k: 36,470 个单元；230k: 230,111 个单元。
角度离散	coarse SI 使用 S_4 ， $M = 24$ ；fine SI、RSI、RSI-tail 使用 S_{128} ， $M = 16640$ 。
随机样本	RSI 使用 8192 条独立样本链；RSI-tail 使用同一批链的 N 到 $N + 10$ 步平均。
CUDA 范围	当前 GPU 路径支持各向同性散射、 $G = 1$ ；Figure 5 的 SI、RSI、RSI-tail 均走 CUDA。
主要代码入口	Figure 5 的 GPU 入口在 <code>RSISolver::runFigure5GPU()</code> ；大角度/大网格自动进入流式 plan 路径。

1. 总体计算流程

Figure 5 中有两类确定性/随机输运计算：

- SI 是确定性角度求积。每轮迭代中，对所有角方向做扫掠，得到角通量，再按权重累加成标量通量 ϕ 。迭代到相邻两轮的相对 L_2 范数小于阈值。
- RSI 是随机方向链。每个样本在每一步随机选一个角方向并扫掠，最终对所有样本的第 N 步结果求均值。RSI-tail 不重新跑链，而是在同一条链上累计第 N 到 $N + 10$ 步。

当前 GPU 路径先上传静态问题数据，包括 mesh、角方向、权重和边界源参数。然后：

1. 先计算 coarse SI。coarse 角度数小，直接使用 SI-only CUDA API。
2. 再计算 fine SI。对 S_{128} 的 16640 个方向按方向块流式处理 sweep plan。
3. fine SI 收敛得到 N 后，生成 RSI 的随机方向调度表。
4. RSI 按样本批执行。每个样本批根据实际用到的去重方向构造紧凑 plan，在 GPU 上推进所有随机链。
5. 最后只把 SI、RSI、RSI-tail 三个最终单元中心场拷回 CPU 写 CSV。

2. Sweep Plan 的作用

非结构四面体网格的扫掠不是简单的规则网格扫描。每个角方向 Ω_m 都需要一个单元顺序：

- 对无环方向，单元依赖图可以拓扑排序。代码保存 order 和 levelOffsets。
- 同一个依赖层内的单元没有上风依赖，可以并行计算。
- 对有环方向，使用投影排序加有限次 Gauss-Seidel pass 作为回退路径。

因此 sweep plan 是 GPU 并行的关键中间结构。它包括：

- orders: 每个方向的单元访问顺序。
- levelOffsets: 无环方向的波前依赖层边界。

例如 `levelCells = [5, 9, 12, 3, 7, 8, 20]`，`levelOffsets = [0, 3, 6, 7]`，表示第 0 层是 `levelCells[0:3] = [5, 9, 12]`，第 1 层是 `levelCells[3:6] = [3, 7, 8]`，第 2 层是 `levelCells[6:7] = [20]`；每一层内部可并行，层与层之间按顺序推进。

- hasCycle: 方向是否需要环回退路径。
- 角方向、权重和局部方向映射。

对 230k + S128，全量 orders 大约是

$$16640 \times 230111 \times 4 \approx 14.7 \text{ GB}$$

仅 sweep order 就接近或超过单卡可用预算，再加场数据、工作区、metadata 会超显存。因此当前实现默认使用流式 sweep plan：一次只把一部分方向的 plan 放到 GPU。

3. SI 的 CUDA 并行方式

3.1 角度方向并行

SI 的天然并行维度是角方向。给定上一轮标量通量 $\phi^{(k-1)}$ ，每个角方向的 sweep 彼此独立。当前实现把方向切成方向块，默认：

$$\text{RSI_CUDA_SI_PLAN_CHUNK} = 128$$

每个方向块进入 GPU 后，执行：

1. 清空当前方向块的临时角通量 angularPsi。
2. 对方向块内方向做 sweep。这里 previousPhi 作为上一轮 SI 的散射源输入，不在当前方向 sweep 中被改写。
3. 每扫完一个方向块，用 accumulateDirectionBatchKernel 把该方向块的角通量按权重累加到当前轮标量通量 currentPhi。
4. 所有方向块完成后，计算相对范数并判断 SI 是否收敛。

也就是说，SI 一轮迭代中的数据流是：

$$\phi^{(k-1)}(\text{previousPhi}) \longrightarrow \psi_m^{(k)}(\text{angularPsi}) \longrightarrow \phi^{(k)}(\text{currentPhi})$$

对每个 cell，累加关系为：

$$\text{currentPhi}[\text{cell}] += w_m \cdot \text{angularPsi}[m, \text{cell}]$$

因此扫完一个方向后，并不是更新上一轮的 previousPhi，而是把这个方向对当前轮新标量通量的贡献累加进去。所有方向贡献累加完以后，currentPhi 才成为下一轮迭代的输入。

这样避免保存完整的 $M \times C$ 角通量，只需要保存一个方向块的临时场和两个标量场缓冲区。

3.2 无环方向的波前并行扫掠

对无环方向，GPU 不再让一个线程串行扫完整个单元序列，而是按依赖层并行：

这里的依赖层，是按当前角方向的上风依赖关系把单元分成的一批一批集合。某个单元如果需要读取上风邻居的角通量，就必须等上风邻居先算完；没有内部上风依赖的单元放在第 0 层，只依赖第 0 层的单元放在第 1 层，依次类推。可以理解为：

$$\text{第 0 层} \rightarrow \text{第 1 层} \rightarrow \text{第 2 层} \rightarrow \dots$$

同一依赖层内的单元彼此没有未满足的上风依赖，所以可以同时计算；不同依赖层之间必须按顺序推进。这就是波前并行扫掠中的波前。

- 代码在 CPU 端按 level=0,1,2,... 逐层 launch kernel；当前处理的是哪一层，由 launch 时传入的 level 参数决定。
- 在普通波前 kernel sweepLevelKernel 中，blockIdx.x 对应方向块里的一个角方向。因此一个 CUDA block 处理一个角方向的当前依赖层。该层里的单元由 block 内线程按 threadIdx.x 分摊。
- 在依赖层内单元分块 kernel sweepLevelTiledKernel 中，grid 是二维的：blockIdx.x 对应一个角方向，blockIdx.y 对应该依赖层里的一个单元分块。因此一个 CUDA block 处理一个角方向、当前依赖层、一个单元分块。
- 这里的分块不是 RSI_CUDA_SI_PLAN_CHUNK=128 那种方向 chunk。方向 chunk 是把很多角方向分批上传和扫掠；这里的分块是把同一个依赖层里的单元再切成多个 cell tile。
- 依赖层之间仍然保持顺序，保证上风依赖已经计算完成；依赖层内单元分块只是在同一依赖层内部增加并行 block 数。

当前主要 kernel 是 sweepLevelKernel / sweepLevelTiledKernel。分块波前并行默认开启：

- RSI_CUDA_SI_WAVEFRONT=1

- RSI_CUDA_SI_TILED_WAVEFRONT=1

当前每个 block 使用 levelSweepThreads=256 个线程。代码先统计当前 plan chunk 中所有方向的最大依赖层宽度 maxSweepLevelWidth，再计算

$$\text{levelTileCount} = \left\lceil \frac{\text{maxSweepLevelWidth}}{256} \right\rceil.$$

如果 RSI_CUDA_SI_TILED_WAVEFRONT=1 且 levelTileCount>1，SI 就使用 sweepLevelTiledKernel；否则使用普通的 sweepLevelKernel。也就是说，只有当某个依赖层宽到 256 个线程一个 block 覆盖不够时，才把依赖层切成多个单元分块。分块的目的不是改变数学算法，而是在依赖层很宽时增加 grid 维度，让更多 SM 同时工作。

3.3 SI 中线程具体做什么

SI 的主路径是无环波前并行扫掠。这里的 GPU 工作分配可以理解为：

- 不分块时，一个 CUDA block 负责一个角方向的当前依赖层；block 内线程负责该层中的不同单元。线程编号从当前依赖层的起点开始，以 blockDim.x 为步长遍历：第 0 个线程处理第 0、256、512... 个单元，第 1 个线程处理第 1、257、513... 个单元。
- 分块时，一个 CUDA block 负责一个角方向的当前依赖层中的一个连续单元片段；通常一个线程处理该片段中的一个单元。多个 blockIdx.y 分块共同覆盖同一个依赖层。
- 每个线程拿到一个单元后，会顺序遍历这个单元的面引用，计算该角方向下每个面的 $\mu = \Omega \cdot n$ 、入流项、出流项和体源项。
- mesh 邻接关系、面几何和材料参数在进入 sweep 前已经上传到 GPU 显存。线程在 kernel 中直接从这些 global memory 数组读取，例如 cellFaceOffsets、refNeighbor、refNx/refNy/refNz、refArea、volume、sigmaT、sigmaS。
- 如果某个入流面有内部邻居，线程用 GPU 上的 refNeighbor 找到邻居 cellid，再从当前方向的 currentPsi 中读取邻居角通量；如果是边界面，则直接调用边界入流函数。整个过程不需要回到 CPU 查询邻接关系。
- 线程最后把该单元在当前角方向下的新角通量写入 angularPsi。

因此，在无环波前路径里，一个线程的核心任务是：计算一个角方向、一个依赖层内若干个单元的有限体积更新。线程之间不直接通信；依赖关系由依赖层顺序推进保证。

3.4 有环方向回退路径

如果某个方向的单元依赖图存在环，代码走 sweepSamplesKernel 回退路径。该 kernel 用投影排序扫单元，并最多做二十次局部迭代。这里的一次完整扫掠，是指按当前排序把这一方向下的所有单元遍历并更新一遍。无环方向只需要一次完整扫掠；有环方向保留最多二十次完整扫掠，用多次局部迭代缓解环形依赖。当前 230k 的高角度主路径大多是无环方向，因此主要性能来自波前并行路径。

回退路径的线程职责不同。一个 warp 有 32 个 lane，代码把它拆成 8 个四线程小组。每个四线程小组处理一个样本当前选中的方向；扫到某个四面体单元时，4 个 lane 分别计算这个单元 4 个面的入流和出流贡献。随后用 warp shuffle 在这 4 个 lane 内把贡献加起来，得到该单元的总入流和总出流，最后由第 0 个 lane 写出这个单元的新角通量。这个路径保留给有环方向或无法分层并行的情况。

3.5 SI 的归约和收敛判断

SI 每轮结束后做两件事：

- accumulateDirectionBatchKernel: 把方向通量按权重累加到 currentPhi。
- relativeNormKernel: 在 GPU 上计算 $\|\phi^{(k)} - \phi^{(k-1)}\|_V^2$ 和 $\|\phi^{(k-1)}\|_V^2$ ，只拷回两个 double 判断收敛。

因此每轮迭代不会把完整场反复拷回 CPU。只有最终 SI 场会 D2H。

4. RSI 的 CUDA 并行方式

4.1 样本链并行

RSI 的天然并行维度是样本。每条样本链独立选择随机方向序列。当前实现先用确定性 RNG 生成调度表：

$$\text{seed} + 17 * M + 0x9e3779b9 * (\text{sample} + 1)$$

这样 CPU/GPU、样本批切分和线程调度不会改变随机链定义。

GPU 执行时按样本批分块。默认单批最多 128 个样本，同时还会根据剩余显存估算样本批容量：

$$\text{bytes per sample} \approx 3C \cdot \text{sizeof}(\text{double}) + (N + \text{tailExtra}) \cdot \text{sizeof}(\text{int})$$

三个 C 长度数组分别用于上一层 ψ 、当前层 ψ 和 RSI-tail 的样本局部累积。

4.2 每批只构造实际需要的方向 plan

S128 有 16640 个方向，但一个 RSI 样本批在 $N + 10$ 步中只会用到其中一部分。当前每个样本批会：

1. 从调度表中收集该样本批实际用到的去重方向。
2. 对这些方向建立 global-to-local 映射。
3. 构造紧凑的 DevicePlanChunk，只上传这批方向需要的扫掠 plan。
4. 把每个迭代步/样本对应的局部方向编号上传到 selectedDirections。

这样 RSI 不需要常驻全量 S128 plan，也不会为当前样本批没用到的方向做 GPU 扫掠。

举一个简化例子。假设当前样本批里只有 3 个样本，每条样本链只看前 4 步：

样本	第 1 步	第 2 步	第 3 步	第 4 步
s_0	17	102	17	401
s_1	88	102	900	17
s_2	401	88	88	102

这里的数字是全局角方向编号。虽然 S128 总共有 16640 个方向，但这个样本批实际只用到了

$$\{17, 88, 102, 401, 900\}$$

这 5 个方向。代码只为这 5 个方向准备 sweep plan，并建立局部编号，例如：

$$17 \rightarrow 0, \quad 88 \rightarrow 1, \quad 102 \rightarrow 2, \quad 401 \rightarrow 3, \quad 900 \rightarrow 4.$$

随后调度表会被改写成局部方向编号上传到 selectedDirections。例如样本 s_0 的方向序列从

$$17, 102, 17, 401$$

变成

$$0, 2, 0, 3.$$

GPU 推进样本 s_0 时，第 1 步用方向 17 的 plan 扫一遍 mesh，得到新的 currentPsi；第 2 步再用方向 102 的 plan，以第 1 步结果作为上一层输入继续扫；第 3 步再次使用方向 17；第 4 步使用方向 401。其它样本也按自己的方向序列独立推进。这样每个样本是一条独立随机链，但同一个样本批共享这批紧凑的方向 plan。

4.3 RSI 波前并行扫掠

对每个 RSI 迭代步，每个样本只选一个方向。但一个方向不是只对应一个编号，而是对应这个方向下整个 mesh 的一次扫掠。对非结构网格，某个方向会把所有 cell 分成多层依赖层：

$$\text{方向 } m \rightarrow \text{第 0 层} \rightarrow \text{第 1 层} \rightarrow \text{第 2 层} \rightarrow \dots$$

同一层内的 cell 可以并行，层与层之间必须按顺序推进。如果这一轮涉及的方向都无环，则 RSI 走波前并行扫掠：

- CPU 端仍然按 level=0,1,2,... 逐层 launch kernel。当前 launch 只处理当前依赖层。
- 不分块时，blockIdx.x 对应样本批里的一个样本。这个 block 处理该样本当前方向的当前依赖层。block 内通常有 256 个线程，线程编号 threadIdx.x 对应该依赖层里的不同 cell：第 0 个线程处理第 0、256、512... 个 cell，第 1 个线程处理第 1、257、513... 个 cell，依次类推。

- 分块时，grid 是二维的：blockIdx.x 仍然对应一个样本，blockIdx.y 对应该依赖层里的一个 cell tile。此时一个 block 只处理当前依赖层中的一段连续 cell，通常一个线程处理这个 tile 里的一个 cell；多个 blockIdx.y 共同覆盖同一个样本、同一个方向、同一个依赖层。
- block 内线程之间不需要交换角通量。每个线程写自己负责的 cell；层间依赖由 CPU 端按 level 顺序 launch kernel 来保证。
- 如果两个样本同一步迭到同一个方向，它们可以共享这个方向的 sweep plan，但 previousPsi/currentPsi 按样本分开存储，所以不会互相覆盖。

RSI 的依赖层内单元分块还多一个样本批条件。只有同时满足以下条件时才分块：

- RSI_CUDA_RSI_TILED_WAVEFRONT=1；
- levelTileCount>1，即最大依赖层宽度超过 256 个单元；
- batchSize>=64，即样本批本身足够大。

否则普通波前并行已经能提供足够的 grid 规模。有环方向单独处理：

- 如果该迭代步中有有环方向，则该迭代步回退到 sweepSamplesKernel。
- 这是保守策略，保证依赖处理简单。

4.4 RSI 与 RSI-tail 合并计算

当前实现不会分别跑 RSI 和 RSI-tail。对每个样本：

- 第 N 步的 currentPsi 进入 globalRSISum。
- 第 N 到 $N+10$ 步的 currentPsi 累加到 sampleTail。
- 一个样本批结束后，再把 sampleTail reduce 到 globalTailSum。

这样 tail 的 11 个时间层只在样本局部工作区中累加，避免每个 tail 迭代步都做一次全局 cell reduction。最终：

$$\phi_{\text{RSI}} = \frac{1}{S} \sum_s \psi_s^{(N)}, \quad \phi_{\text{tail}} = \frac{1}{S(10+1)} \sum_s \sum_{j=0}^{10} \psi_s^{(N+j)}$$

5. 显存与流式 Plan 策略

当前入口会估算全量 orders 的显存：

$$M \times C \times \text{sizeof}(\text{int})$$

如果超过阈值，默认 8 GiB，就走流式 sweep plan。该阈值可用：

RSI_CUDA_STREAMING_PLAN_THRESHOLD_MB

调整。

流式 sweep plan 的核心逻辑是：

- mesh、ordinates、weights 等静态数据只上传一次。
- SI 按固定方向块顺序处理。
- RSI 按样本批实际用到的去重方向临时组装紧凑 plan。
- plan 不直接等同于场数据；场数据工作区与 plan 工作区分开管理。

这使得 230k + S128 + 8192 样本可以在 16 GB 级别 GPU 上完整运行。

6. 缓存与 Plan 优化逻辑

6.1 磁盘方向块缓存

每个角方向在 sweep 前都要先生成一个 sweep plan。它记录这个方向下 GPU 扫掠需要的结构信息，包括：

- cell 应该按什么顺序扫；
- 该方向的依赖图是否有环；
- 无环方向有哪些依赖层；
- 每个依赖层在 sweep order 中从哪里开始、到哪里结束。

生成这些 plan 需要遍历 mesh 的邻接关系。对 230k + S128 来说，方向数达到 16640，如果每次运行都重新构建所有方向的 plan，开销会很大。因此当前实现把已经生成好的方向块 plan 序列化到磁盘：

```
results/cache/rsi_sweep_plan_chunk_<key>.bin
```

下次遇到同样的 mesh、同样的角方向集合和同样的缓存格式版本时，就直接从这个 .bin 文件读回 plan，不重新构建。cache key 由 mesh、角方向、方向列表和格式版本决定；如果 mesh 变了、方向集合变了，或者缓存文件损坏/版本不匹配，代码会忽略旧缓存并重新构建。

SI 和 RSI 的缓存粒度不同。SI 每轮本来就要顺序扫完整的方向集合，因此用较大的方向块比较合适；当前默认 RSI_CUDA_SI_PLAN_CHUNK=128，S128 的 16640 个方向大约对应 130 个 SI 方向块缓存。RSI 则不同，每个样本批只随机用到其中一部分方向。例如 128 个样本的 batch 往往只用到约 2800 个去重方向，而且这些方向随机分布在 0 到 16639 之间。如果 RSI 也按 128 个方向一块缓存，一个 batch 只要碰到某个 chunk 中的少数方向，就可能需要读取整个 128-direction chunk，最后接近读取全量 S128 plan，包含大量本批没用到的方向。

因此当前 RSI 固定方向缓存默认使用

```
RSI_CUDA_FIXED_PLAN_CHUNK=1
```

也就是一个缓存文件对应一个方向。这样 S128 最多约 16640 个方向缓存文件，但每个样本批只加载自己实际用到的方向，避免随机访问时读入大量无用 plan。

6.2 主机端 LRU 缓存

磁盘缓存解决的是“不要重新生成 plan”，但它没有消除读文件本身的开销。如果某个方向块每次都从 .bin 文件取，仍然要经历：

1. 打开缓存文件；
2. 读取二进制内容；
3. 检查 header、方向数和 cell 数；
4. 反序列化出每个方向的 order、levelOffsets 和 hasCycle；
5. 再把这些 SweepPlan 打包成 GPU 上传需要的 HostPlanChunk。

所以当前实现又加了一层 CPU 内存缓存，也就是主机端 LRU 缓存：

```
RSI_CUDA_PLAN_HOST_CACHE_MB
```

默认 1024 MiB。这里缓存的不是 GPU 设备指针，而是主机内存里的 vector<SweepPlan>。缓存键和磁盘 .bin 文件使用同一个方向块 key；也就是说，同一个 mesh、同一个方向列表、同一个缓存格式版本，会查到同一个主机端缓存项。

一次方向块准备的查找顺序是：

1. 先用 cache key 查主机端 LRU。
2. 如果命中，直接复制内存中的 vector<SweepPlan>，跳过磁盘读取和反序列化。
3. 如果没有命中，再去读对应的 .bin 文件。
4. 如果磁盘也没有命中，才重新遍历 mesh 邻接关系构建 sweep plan。
5. 从磁盘读到或者重新构建出的 plan，会再放回主机端 LRU，供后续方向块复用。

LRU 是 least recently used，中文可以理解成“最近最少使用”。当缓存占用超过预算时，代码优先丢掉最久没有被访问的方向块，保留最近反复用到的方向块。这样做的原因是 SI 和 RSI 都有明显的重复访问：

- SI 每轮迭代都按相同方向块顺序扫完整角方向集合。第 1 轮把方向块放入 LRU 后，后面的 SI 轮次再次访问同一个 chunk 时，通常可以直接从主机内存拿到 plan。
- RSI 每个样本批只使用一部分随机方向，但不同样本批之间会反复碰到相同的固定方向。单方向固定缓存配合主机端 LRU 后，后续 batch 再需要这个方向时，也可以避免重复读磁盘。

需要注意的是，主机端 LRU 只减少 CPU 侧准备 plan 的开销；它本身不等于 GPU 端常驻。命中 LRU 后，代码仍然要把这些 plan 重新组装成 HostPlanChunk，并在需要时上传成 DevicePlanChunk。如果想进一步避免 SI 每轮重复 H2D 上传，就要靠下一节的 SI 设备端 plan 缓存。

因此缓存层级可以理解为：

重新构建 plan > 读磁盘.bin > 主机端 LRU 命中 > GPU 设备端常驻

越往右复用成本越低。

6.3 SI 设备端 plan 缓存

SI 每轮迭代都会按同一组角方向、同一个方向块顺序重复扫掠。例如 S128 使用 RSI_CUDA_SI_PLAN_CHUNK=128 时，16640 个方向会被切成约 130 个方向块。第 1 轮 SI 要扫这 130 个方向块；第 2 轮、第 3 轮也还是按同样顺序再扫同一批方向块。

如果只有 6.2 的主机端 LRU，那么第 2 轮以后可以少读磁盘，但仍然要把每个方向块从 CPU 侧组装并上传到 GPU。对 SI 来说，这个上传会在每一轮重复发生。因此当前实现又提供了一层 SI 专用的设备端 plan 缓存：

RSI_CUDA_SI_DEVICE_PLAN_CACHE_MB

默认 8192 MiB。这里缓存的是已经上传到 GPU 的 DevicePlanChunk，包括该方向块在 GPU sweep 中要用的方向编号、方向余弦、权重、cell 扫掠顺序、依赖层 offset 等数组。

一次 SI 方向块的处理流程是：

1. CPU 按 chunkIndex 访问当前方向块。
2. 先检查 siDevicePlanCache[chunkIndex] 是否已经有对应的 DevicePlanChunk。
3. 如果命中，直接用 GPU 上已有的 plan launch sweep kernel，不需要重新上传这个方向块的 plan。
4. 如果没有命中，就走 6.2 的主机端准备流程：查 LRU、查磁盘，必要时重新构建 SweepPlan。
5. 主机端 plan 准备好后，上传成 DevicePlanChunk。
6. 如果该方向块不超过设备端缓存预算，并且剩余预算够用，就把它留在 siDevicePlanCache 对应位置；否则只临时使用，用完释放。

因此第 1 轮 SI 通常会产生较多 cache miss，因为方向块第一次出现时还没在 GPU 上。只要显存预算够，代码会把这些上传过的方向块留下。第 2 轮以后再次访问相同 chunkIndex 时，就变成 cache hit，直接复用 GPU 内存里的 plan。

这个策略主要适合 SI，而不是 RSI，原因是 SI 的方向访问顺序固定并且重复很多轮；RSI 每个样本批的随机方向集合不同，设备端常驻大块 plan 的命中规律没有 SI 稳定。RSI 主要依赖 6.4 的固定方向缓存和 6.5 的预热/并行加载来降低 plan 准备开销。

简单说，三层缓存各自解决的问题不同：

- 磁盘缓存：避免重新遍历 mesh 构建 sweep plan。
- 主机端 LRU：避免反复读 .bin 和反序列化。
- SI 设备端缓存：避免 SI 每轮把同一个方向块 plan 反复从 CPU 上传到 GPU。

6.4 RSI 固定方向缓存

RSI 的方向访问模式和 SI 不一样。SI 是固定顺序反复扫完整角方向集合；RSI 是每个样本链按调度表随机选方向。以 S128、8192 个样本为例，总方向数是 16640，但每个 128-sample batch 通常只用到约 2800 个去重方向，而且这些方向在 0 到 16639 之间随机分布。

如果 RSI 直接按 128 个方向一个 chunk 来复用 plan，会有一个问题：某个 batch 可能只需要 chunk 里的少数几个方向，却必须把整个 128-direction chunk 都读出来、打包、上传。因为随机方向覆盖范围很散，一个 batch 最后可能触碰很多 chunk，等价于加载大量本批不用的方向。

所以当前 RSI 使用固定方向缓存，把大方向块拆成更小的固定方向块。默认：

```
RSI_CUDA_FIXED_PLAN_CHUNK = 1
```

也就是一个固定缓存块只包含一个全局角方向。对 S128 来说，最多约 16640 个固定方向缓存文件。这样文件数量变多，但每个样本批只加载自己真正用到的方向，避免随机访问时把大量无关方向带进来。

一个 RSI batch 准备 plan 时，大致流程是：

1. 从该 batch 的所有样本链和所有迭代步中收集全局方向编号。
2. 排序并去重，得到 uniqueDirections。
3. 对每个需要的全局方向，找到它所属的固定方向缓存块。
4. 对已经有缓存的方向块，直接读固定方向 plan。
5. 对没有缓存的方向块，重新构建对应方向的 SweepPlan，然后保存到磁盘缓存。
6. 把这些固定方向 plan 按当前 batch 的 uniqueDirections 顺序重新组装，形成紧凑的 HostPlanChunk。
7. 再上传成当前 batch 使用的 DevicePlanChunk。

这里要注意，固定方向缓存解决的是“方向 plan 的复用”，不是让每个方向都常驻 GPU。RSI 每个 batch 仍然会构造一个只包含本批去重方向的紧凑 DevicePlanChunk。这样 GPU sweep 时只面对当前 batch 实际需要的方向，内存占用比全量 S128 plan 小得多。

日志里的 rsi_plan_fixed_cache_hits 和 rsi_plan_fixed_cache_misses 就是在看这层固定方向缓存的效果。hit 表示这个固定方向 plan 已经存在，可以复用；miss 表示这个方向还没有缓存，需要构建并保存。

6.5 RSI 预热与并行加载

固定方向缓存能减少重复构建，但如果等到每个 batch 真正运行时才发现某个方向 miss，就会把构建 plan 的开销插入到 batch 准备阶段。为了减少运行中突然 miss，当前 RSI 在正式 batch sweep 前可以做预热：

```
RSI_CUDA_RSI_PLAN_PREWARM=1
```

默认开启。预热的前提是 RSI 方向调度表已经生成。调度表生成后，代码已经知道所有样本、所有迭代步会访问哪些全局方向，因此可以在正式 sweep 前先扫描完整调度表。这里得到的不是某一个 batch 的 uniqueDirections，而是整次 run 会用到的全局方向集合，再把这些方向映射成固定方向块集合。

如果 RSI_CUDA_FIXED_PLAN_CHUNK=1，一个固定方向块正好就是一个全局方向，所以“固定方向块集合”和“整次 run 的去重方向集合”基本等价。如果固定方向块大小大于 1，一个固定方向块会包含多个连续全局方向；只要其中任意一个方向被本次 run 用到，这个固定方向块就会进入预热集合。

预热阶段的流程是：

1. 扫描完整 RSI 调度表，先得到整次 run 的去重全局方向集合。
2. 把这些全局方向映射到固定方向块，得到需要预热的固定方向块集合。
3. 对每个需要预热的固定方向块，先检查对应的磁盘缓存文件是否已经存在。
4. 如果文件已经存在，只记录为 prewarm hit，不重新构建。
5. 如果文件不存在，记录为 prewarm miss，并构建这个固定方向的 sweep plan。
6. 构建完成后保存为固定方向缓存文件，后续 batch 直接读取。

这一步的目的不是提前把所有 plan 放进 GPU，而是提前补齐磁盘/主机侧可复用的固定方向 plan。这样 batch 阶段更可能只做读取、组装和上传，而不是边运行边构建。

预热和固定方向加载可以并行执行：

```
RSI_CUDA_FIXED_PLAN_PARALLEL_LOAD=1
```

默认开启。实际含义是用多个 CPU worker 同时处理多个固定方向块：有缓存的方向并行读取；缺缓存的方向并行构建并保存。预热 worker 数可以由 RSI_CUDA_FIXED_PLAN_PREWARM_WORKERS 控制；如果没有单独设置，通常复用固定方向加载 worker 的设置。

还需要 miss 去重。因为多个 batch 或多个 worker 可能同时发现同一个固定方向缺缓存，如果不去重，就可能有多个线程重复构建同一个方向 plan。当前逻辑默认开启 miss 去重：

```
RSI_CUDA_FIXED_PLAN_MISS_DEDUP=1
```

含义是：同一个固定方向块 miss 时，只让一个 worker 负责构建和保存，其它 worker 等待；等缓存文件生成后，其它 worker 再读取结果。这样减少重复构建，也避免多个线程同时写同一个缓存文件。

因此 6.4 和 6.5 配合起来看，RSI plan 优化分成两步：

- 固定方向缓存：把随机方向访问拆成可复用的单方向 plan，避免读入无关方向。
- 预热与并行加载：在正式 batch sweep 前尽量补齐这些单方向 plan，并把缺失构建并行化。

以现有 230k + S128 + 8192 日志为例，预热阶段输出：

```
CUDA RSI fixed plan prewarm: chunks=16640, hits=16639, misses=1
```

说明这次运行前 S128 的固定方向缓存基本已经齐全，正式 RSI batch 中主要成本就转移到固定方向 plan 的读取、组装、上传和真正 sweep 上，而不是大量重新构建 sweep plan。

6.6 RSI fused wavefront sweep

6.4 和 6.5 主要降低 RSI 的 plan 准备成本；当固定方向缓存已经齐全以后，230k + S128 + 8192 的主开销又回到真正的 sweep kernel。当前 RSI 对无环方向提供 fused wavefront 路径：

```
RSI_CUDA_RSI_FUSED_WAVEFRONT = 1
```

每个 RSI batch 包含若干条样本链，每个样本在当前 iteration 只选择一个方向。对无环方向，当前 fused 路径把该样本当前方向的所有依赖层放在同一个 kernel 内部顺序推进，避免在 CPU 端为每一层单独组织一次 launch。230k 网格的每个方向通常有数百个依赖层；对 8192 样本、 $N + 10$ 个随机步来说，level 组织方式会直接影响 sweep 阶段的调度成本和尾部并行度。

该路径复用已有的 sweepLevelFusedKernel。一个 CUDA block 仍然对应一个样本；block 内线程处理该样本当前方向的一个依赖层。不同的是，kernel 内部按 level 顺序循环：

```
level=0 → level=1 → ...
```

每层结束后用 __syncthreads() 保证同一个样本、同一个方向内部的上风依赖已经完成，然后继续下一层。这样一次 kernel launch 就覆盖一个样本当前方向的一整次无环 sweep，而不是每个 level 单独 launch。

这个 fused 路径只用于无环 RSI wavefront；如果某个 iteration 中存在有环方向，仍然回退到 sweepSamplesKernel 多 pass 路径。当前默认策略是：

- 当 cell 数 $C \geq 100000$ 时默认启用 fused RSI wavefront，因为大网格的 level 数多，减少 launch 数更有收益。
- 小网格默认保留 tiled level-by-level 路径，避免在 level 数较少时引入不必要的 fused kernel 组织成本。
- 可用 RSI_CUDA_RSI_FUSED_WAVEFRONT=0 显式关闭，便于定位性能或数值问题。

从工作分配看，fused RSI wavefront 没有改变随机链、方向调度表、sweep plan、边界源或离散方程。它只改变同一个样本当前方向内部 level 的 launch 组织方式：

- level-by-level 路径：CPU 端逐 level launch，level 间顺序由 kernel launch 顺序保证。
- fused 路径：GPU kernel 内逐 level 循环，level 间顺序由同一 block 内的 __syncthreads() 保证。

因此 fused 路径和 level-by-level 路径应当给出逐值一致的结果。实际验证中，230k + S64 + 512 的 SI_fine.csv、RSI.csv 和 RSI_tail.csv 在 fused 开关前后逐字节一致；小规模 gpu_consistency_test 的 CPU/GPU 相对 L_2 误差约为 10^{-16} 。

在当前 230k + S128 + 8192 streaming 日志中，RSI fused wavefront 运行在 S128 RSI sweep 阶段，当前口径为：

指标	total / s	plan / s	sweep / s
RSI	701.386	97.775	566.236

这说明当前完整 Figure 5 时间中，RSI 仍主要由 sweep 本身和 plan 组装/上传组成。fused wavefront 覆盖的是无环 RSI sweep 的 level 组织方式；它不处理 SI sweep，也不消除 RSI plan 的固定方向读取、assemble、pack 和 upload 成本。

这次也尝试过把 $\sigma_T \times \text{volume}$ 、 $\sigma_S \times \text{volume}$ 和 $q \times \text{volume}$ 预计算到 GPU mesh 数组中，减少每个 cell 更新里的乘法。这里的 A/B 指两个实现变体：A 为当前保留路径，在 sweep kernel 内按需计算这些乘积；B 为实验路径，在 mesh 上传阶段预先生成乘积数组，sweep kernel 内直接读取。230k + S16 + 16 的快速验证结果波动较大，fine SI sweep 没有稳定改善，甚至有小幅回归。因此该方案没有保留。当前判断是 sweep kernel 更受 wavefront 并行度、global memory 访问和 kernel launch 组织影响，单元公式中的少量算术不是主要瓶颈。

7. Mesh 数据布局优化

这一节的核心是：把 sweep 计算 cell 时需要的 mesh 数据，提前整理成 GPU 友好的数组，并一次性上传到 GPU global memory。线程处理 cell 时直接读取 volume、 σ_T 、 σ_S 、cellQ、cellFaceOffsets、refNeighbor、refNx/refNy/refNz、refArea 和边界数组，不需要运行中回到 CPU 查询 mesh。

当前 GPU mesh 布局类似 CSR：cellFaceOffsets[cell] 到 cellFaceOffsets[cell+1] 给出该 cell 的面引用区间；每个 refIndex 在多条 SoA 数组中保存邻居、法向、面积、面中心和边界信息。这样 kernel 不必从 cell 再追到 face 对象，减少了一层间接索引，也降低了 GPU 端对象结构开销。

需要注意的是，这并不把非结构网格访存变成规则连续访问。不同线程处理的 cell 在 sweep order 中不一定相邻，读取 currentPsi[neighbor] 仍可能是随机位置。这个优化的作用是把几何、邻接和边界数据放在 GPU 上并按面引用对齐，让 SI、RSI 和 RSI-tail 共用同一份 GPU mesh 数据。

8. 当前性能口径

当前 Figure 5 日志包含两组网格、两组角度口径。S4 是 coarse SI， $M = 24$ ，只做 SI-only；S128 是 fine SI + RSI + RSI-tail， $M = 16640$ ，RSI 使用 8192 条样本链。下面按 30k non-streaming、30k streaming 和 230k streaming 三种当前运行路径列出日志口径。

8.0 计时字段说明

字段	含义
key	计算 sweep plan 缓存 key 和缓存文件路径的时间。
cache	从已有 plan cache 读取并反序列化的时间；RSI fixed cache 中这是多线程累计时间，不一定等于 wall time。
build	cache miss 时从 mesh 和角方向重新构建 SweepPlan 的时间，包括计算 order、levelOffsets 和 hasCycle。
save	把新构建的 sweep plan 写入磁盘 cache 的时间。
assemble	把固定方向 plan 按当前 batch 的 uniqueDirections 顺序组装成紧凑 plan 的时间，主要用于 RSI。
pack	把 SweepPlan 打包成可上传的 HostPlanChunk 数组布局的时间。
upload	把 mesh、方向、plan 或工作数组上传到 GPU 的时间，具体含义随日志阶段而定。
sweep	GPU 上真正执行输运 sweep kernel 的时间，即按方向/样本和依赖层计算 cell 通量。
schedule	生成 RSI 随机方向调度表的时间。
unique	每个 RSI sample batch 从调度表中收集、排序并去重实际用到方向的时间。
reduce/norm/copy	分别是结果归约、SI 收敛范数计算和最终 GPU 到 CPU 拷贝时间。

8.1 30k non-streaming 计时

角度	方向	plan/setup / s	SI total / s	SI sweep / s	RSI total / s	RSI sweep / s
S4	24	0.160	0.219	0.049	—	—
S128 non-streaming	16640	1.738	60.139	58.034	88.500	84.712

这里的 plan/setup 采用 CUDA 日志中的 upload 口径。non-streaming 的 S128 全量 plan 可以放入显存，fine SI 和 RSI/RSI-tail 共享同一套 GPU 问题数据；日志不会把这部分拆成独立的 SI plan 和 RSI plan。

为了测量从零准备完整 sweep plan 的时间，另用临时缓存目录

```
RSI_SWEEP_PLAN_CACHE_DIR=/tmp/rsi_sweep_cache_cold_30k_nonstream
```

重新跑了一次 30k non-streaming。此时原有 results/cache 不参与命中，日志中的完整 plan 准备时间为：

阶段	方向	build / s	save / s	说明
coarse SI plan	24	0.010	0.003	构建并缓存 S4 sweep plan。
fine SI/RSI plan	16640	9.203	2.742	构建并缓存 S128 全量 sweep plan。

同一次冷缓存运行中，S128 GPU problem 上传 upload=1.847 s，RSI 随机方向调度表生成 rsi_schedule=0.485 s，外部 wall time 为 elapsed 159.77 s。因此对 30k non-streaming 来说，一开始完整计算扫描顺序和依赖层的主要成本就是上表的 build，而不是后续 batch 中的 fixed-direction miss。

30k non-streaming 这次完整命令的外部 wall time 为：

elapsed 151.05 s

8.2 30k streaming 计时

为了让 30k 也走和 230k 相同的 streaming 逻辑，强制设置

RSI_CUDA_STREAMING_PLAN_THRESHOLD_MB=0

跑了一次 30k S128 streaming。这个路径会像 230k 一样把 SI 的 16640 个方向组织成 130 个 128-direction chunk，并让 RSI 每个 batch 构造紧凑 plan。

角度	方向	plan/setup / s	SI total / s	SI sweep / s	RSI total / s	RSI sweep / s
S4	24	0.148	0.204	0.046	–	–
S128 streaming	16640	19.299	60.965	52.591	102.494	84.747

30k streaming 的 plan/setup 可以拆成：

阶段	total / s	key / s	cache / s	assemble / s	pack / s	upload / s
SI plan	6.356	0.008	5.169	0.000	0.156	0.880
RSI plan	12.943	1.137	130.667	3.841	1.734	8.289

这里 30k streaming 的 plan/setup 为 19.299 s，由 6.356 s 的 SI plan 和 12.943 s 的 RSI plan 组成。其中 RSI plan 的 cache 计时为 130.667 s，这是多线程固定方向缓存读取的累计计时，不等于 wall time；日志中对应的固定方向缓存 wall time 是 8.577 s。

30k streaming 这次完整命令的外部 wall time 为：

elapsed 158.44 s

30k streaming 中 SI 的 130 个 plan chunk 全部常驻设备端缓存：

device chunks=130/130, hits=1690, misses=130

因此 30k streaming 的 SI sweep 主要受 GPU sweep kernel 本身影响；RSI 仍需要按样本批读取固定方向 plan、组装紧凑 plan 并上传。

8.3 230k streaming 计时

角度	方向	plan/setup / s	SI total / s	SI sweep / s	RSI total / s	RSI sweep / s
S4	24	0.305	0.565	0.228	–	–
S128 streaming	16640	145.147	801.394	720.885	701.386	566.236

230k 的 S128 走 streaming plan 路径，因此日志可以单独统计 SI plan 和 RSI plan。本机约 16 GB host memory 下，S128 + 8192 使用

RSI_CUDA_RSI_MAX_SAMPLES_PER_BATCH=64

限制 RSI batch 大小，避免 host 侧 batch plan/prefetch 内存峰值过高。

阶段	total / s	key / s	cache / s	assemble / s	pack / s	upload / s
SI plan	47.372	0.046	16.079	0.000	1.480	24.122
RSI plan	97.775	3.557	955.513	41.186	10.759	53.701

注意这里的 RSI plan cache=955.513 s 是多次固定方向缓存读取的累计计时，不等于 wall time；日志中对应的固定方向缓存 wall time 是 64.368 s。230k streaming 中 SI 设备端缓存不能容纳全部 130 个 chunk，本次日志为：

device chunks=88/130, host chunks=36/130

230k streaming 这次完整命令的外部 wall time 为：

elapsed 1437.14 s

综合这三个小节可以看到：S4 coarse SI 很快；S128 下真正占主导的是 sweep。30k non-streaming 的 S128 中，SI sweep 约 58.0 s，RSI sweep 约 84.7 s；30k streaming 的 S128 中，SI sweep 约 52.6 s，RSI sweep 约 84.7 s；230k streaming 的 S128 中，SI sweep 约 720.9 s，RSI sweep 约 566.2 s。tail 累加、最终 reduce、方向调度和 D2H copy 都不是主瓶颈。plan 相关成本已经通过缓存、预热和 streaming plan 控制在可运行范围内，但 230k + S128 的 plan 读取/组装/上传仍然是可见开销。

附录 A：历史 27 组 RSI/SI 并行效率测试

本附录保留早期验证数据；当前完整因子矩阵及同网格参考误差定义见后面的 8.4。

以下两表覆盖全部 27 个已完成的空间-角度-样本-batch 组合。新增 S100（10,200 个方向）在 10k/30k/100k 三种网格上以 5,000 样本、 $B = 128$ 复测，覆盖约 10^2 、 10^3 、 10^4 三个角度规模；10k/30k 的 S10-S30 历史日志也已纳入。100k/S224 的 100-100,000 样本与 $B = 32/128/256$ 扫描保留，用于分离样本数和 batch 的影响。网格标签后的数值是实际 cell 数量的约数：10k/9k、30k/36k、100k/94k。 B 为实际 batch capacity；‘-’ 表示旧日志未记录该字段。

表中 error 为 RSI-tail 最终场相对同一空间网格、同一 rep=1 的确定性 SI 最终场的相对 L_2 误差。每个参考场均采用 S316，即 100,488 个方向；参考 SI 的具体配置如下。SI 在 RSI 前独立完成，故下表的关联样本数和 B 只是同一日志中随后 RSI 的设置，不会影响参考 SI 场。

参考空间网格/cells	参考 SI 角度/方向数	关联样本数	B				
10k/9k	S316 / 100488	5000	128				
30k/36k	S316 / 100488	5000	128				
100k/94k	S316 / 100488	5000	-				

网格/cells	角度/方向数	样本数	B	SI total/s	RSI total/s	RSI/SI	error
10k/9k	120	5000	128	0.042	2.276	0.02x	12.556%
10k/9k	440	5000	128	0.149	2.831	0.05x	6.497%
10k/9k	960	5000	128	0.285	3.350	0.08x	4.533%
10k/9k	5040	1000	-	1.634	1.770	0.92x	3.882%
10k/9k	5040	5000	128	1.094	4.845	0.23x	1.920%
10k/9k	10200	5000	128	2.491	5.763	0.43x	1.796%
10k/9k	50624	5000	128	14.572	14.031	1.04x	0.956%
10k/9k	100488	5000	128	29.554	18.841	1.57x	0.842%
30k/36k	120	5000	128	0.143	5.933	0.02x	11.211%
30k/36k	440	5000	128	0.457	6.897	0.07x	6.037%
30k/36k	960	5000	128	18.495	256.681	0.07x	4.132%
30k/36k	10200	5000	128	44.393	65.697	0.68x	1.680%
30k/36k	50624	5000	128	169.717	63.580	2.67x	0.888%
30k/36k	100488	5000	128	399.769	75.389	5.30x	0.757%
100k/94k	10200	5000	128	14.666	40.123	0.37x	1.892%
100k/94k	50624	100	100	136.412	1.694	80.51x	14.289%
100k/94k	50624	500	32	133.890	6.225	21.51x	5.475%
100k/94k	50624	500	128	136.112	6.263	21.73x	5.475%
100k/94k	50624	500	256	142.403	8.213	17.34x	5.475%
100k/94k	50624	1000	128	136.483	11.034	12.37x	2.697%
100k/94k	50624	5000	-	221.185	133.404	1.66x	1.062%
100k/94k	50624	5000	128	138.568	49.903	2.78x	1.062%
100k/94k	50624	10000	128	135.631	117.637	1.15x	0.782%
100k/94k	50624	50000	128	141.423	499.754	0.28x	0.344%
100k/94k	50624	100000	128	135.101	980.555	0.14x	0.315%
100k/94k	100488	5000	-	719.068	246.544	2.92x	0.902%
100k/94k	100488	10000	128	639.425	168.518	3.79x	0.682%

表 1 显示，低角度下 RSI 的 batch、随机调度和 plan 开销不能摊销；120 个方向时三种网格均慢于 SI。高角度组合则出现总时间收益，例如 30k/S316/5,000 样本为 5.30x（0.757%），100k/S316/10,000 样本为 3.79x（0.682%）。S100 的总时间均使用 fixed-plan cache 预热后的复测结果，避免把首次建 cache 的成本混入网格间对比。

两种方法按同一个 directional cell-sweep 工作单位计算并行效率，单位均为 Mcell-sweeps/s：

$$E_{SI} = \frac{\text{cells} \times N_{\Omega} \times N}{t_{SI, \text{sweep}} \times 10^6}, \quad E_{RSI} = \frac{\text{cells} \times N_s \times (N + 10)}{t_{RSI, \text{sweep}} \times 10^6}.$$

其中 N_{Ω} 为角度/方向数， N_s 为 RSI 样本数， N 是 SI 收敛迭代数， $N + 10$ 对应 RSI-tail 共用链的步数。 E_{RSI}/E_{SI} 是在同一工作单位下 RSI 相对 SI 的 GPU 并行效率提升，而不是端到端时间加速比；plan/s 保留以识别准备和传输成本。

网格/cells	角度/方向数	样本数	B	SI sweep/s	E_{SI}	RSI sweep/s	E_{RSI}	E_{RSI}/E_{SI}	plan/s
10k/9k	120	5000	128	0.011	1449.98	1.892	582.65	0.40x	0.110
10k/9k	440	5000	128	0.041	1380.92	2.066	533.62	0.39x	0.459
10k/9k	960	5000	128	0.091	1353.64	2.107	523.20	0.39x	0.880
10k/9k	5040	1000	-	0.436	1486.32	0.420	525.10	0.35x	1.251
10k/9k	5040	5000	128	0.428	1514.14	2.163	509.61	0.34x	2.159
10k/9k	10200	5000	128	0.985	1331.96	2.125	518.77	0.39x	3.105
10k/9k	50624	5000	128	4.256	1530.02	1.846	597.08	0.39x	10.874
10k/9k	100488	5000	128	8.595	1503.73	2.216	497.52	0.33x	14.339
30k/36k	120	5000	128	0.034	1821.49	5.039	868.47	0.48x	0.281
30k/36k	440	5000	128	0.125	1795.60	4.979	878.95	0.49x	1.187
30k/36k	960	5000	128	16.623	29.49	237.872	18.40	0.62x	2.461
30k/36k	10200	5000	128	39.130	133.09	56.585	77.34	0.58x	7.149
30k/36k	50624	5000	128	124.844	207.04	35.913	121.86	0.59x	23.372
30k/36k	100488	5000	128	268.417	191.15	38.993	112.24	0.59x	30.913
100k/94k	10200	5000	128	7.789	1730.33	10.031	1128.98	0.65x	27.531
100k/94k	50624	100	100	37.299	1793.31	0.211	1074.85	0.60x	1.400
100k/94k	50624	500	32	36.661	1824.52	3.022	374.70	0.21x	2.559
100k/94k	50624	500	128	37.288	1793.87	0.982	1153.24	0.64x	4.864
100k/94k	50624	500	256	39.112	1710.18	0.944	1199.83	0.70x	6.798
100k/94k	50624	1000	128	37.608	1778.59	1.865	1214.39	0.68x	8.335
100k/94k	50624	5000	-	35.814	1867.66	9.579	1182.32	0.63x	119.365
100k/94k	50624	5000	128	37.333	1791.67	9.528	1188.58	0.66x	36.243
100k/94k	50624	10000	128	36.716	1821.81	19.334	1171.55	0.64x	88.920
100k/94k	50624	50000	128	37.176	1799.24	96.469	1173.99	0.65x	350.826
100k/94k	50624	100000	128	36.404	1837.43	190.733	1187.56	0.65x	685.353
100k/94k	100488	5000	-	72.743	1825.25	10.039	1128.19	0.62x	230.564
100k/94k	100488	10000	128	120.953	1097.73	19.598	1155.78	1.05x	137.326

大网格下 SI 的 E_{SI} 为约 1.1–1.9 Gcell-sweeps/s，RSI 的 E_{RSI} 通常为约 1.1–1.2 Gcell-sweeps/s，因此在相同 sweep 工作单位下 RSI 的并行效率为 SI 的 0.58–0.70x； $B = 32$ 会进一步降至 0.21x。 S_{316} 、10,000 样本点为 1.05x，属于测量波动下的接近持平。RSI 的端到端优势来自减少必须计算的方向/样本总工作量，而不是单个 sweep 比 SI 更高效。代表性 NCU profile：256 threads/block、achieved occupancy 29.84%、L2 hit 96.47%、L1/TEX hit 27.68%、DRAM throughput 3.05% peak。

8.4 网格、角度、样本与 batch 因子的 270 组测试

本节原计划覆盖三种空间网格（10k、30k、100k）、六档方向数（120、440、960、5,040、10,200、50,624）、五档样本数（8、64、512、1,024、4,096）和三档请求 batch（64、128、256），共 $3 \times 6 \times 5 \times 3 = 270$ 组。当前完成 269 组：10k 与 30k 各 90 组，100k 完成 89 组；唯一缺少 100k/ S_{224} /4,096 样本/请求 $B = 256$ 。每张表按方向数、样本数、请求 batch 由少到多排列；实际 B 为运行时实际容量，小于请求 B 时表示样本数本身更小。

每种空间网格均使用本网格上的确定性 SI 场作为参考解，均固定为 S_{316} （100,488 个方向）、8,192 样本、请求/实际 $B = 128$ ；其中样本数和 batch 仅为完整 Figure 5 运行配置，参考场本身取 SI fine 输出。三个参考 case 如下：

空间网格/cells	参考 SI 配置	case
10k/9,187	S_{316} / 100,488 directions / 8,192 / $B = 128$	10k_s316_r8192_b128_rep1
30k/36,470	S_{316} / 100,488 directions / 8,192 / $B = 128$	30k_s316_r8192_b128_rep1
100k/94,378	S_{316} / 100,488 directions / 8,192 / $B = 128$	100k_s316_r8192_b128_rep1

因此，每个 RSI-tail error 都是在相同空间网格、相同 cell 顺序上相对该网格 S_{316} SI 场计算的相对 L_2 误差；不再进行跨网格插值或最近邻映射。error 可直接用于解释 RSI 的随机样本、角度离散和 batch 因子，而不混入空间网格差异。

两种方法的并行效率使用同一 directional cell-sweep 单位（Mcell-sweeps/s）：

$$E_{SI} = \frac{\text{cells} \times N_{\Omega} \times N}{t_{SI, \text{sweep}} \times 10^6}, \quad E_{RSI} = \frac{\text{cells} \times N_s \times (N + 10)}{t_{RSI, \text{sweep}} \times 10^6}.$$

其中 N_{Ω} 是方向数， N_s 是样本数， N 是 SI 收敛迭代数。效率表中的 RSI/SI 是 E_{RSI}/E_{SI} ，即同一 sweep 工作单位下 RSI 相对 SI 的并行吞吐提升倍数；时间表中的 RSI/SI 则为 $t_{SI, \text{total}}/t_{RSI, \text{total}}$ ，即端到端加速比。

Table 1: 10k 网格 (9187 cells): 端到端时间与同网格 S316 参考误差

方向数	样本数	请求 B	实际 B	SI total/s	RSI total/s	RSI/SI	error (%)
120	8	64	8	0.037	0.057	0.65x	28.411
120	8	128	8	0.026	0.048	0.54x	28.411
120	8	256	8	0.028	0.067	0.42x	28.411
120	64	64	64	0.023	0.043	0.54x	15.044
120	64	128	64	0.026	0.047	0.55x	15.044
120	64	256	64	0.023	0.045	0.51x	15.044
120	512	64	64	0.028	0.396	0.07x	12.985
120	512	128	128	0.024	0.272	0.09x	12.985
120	512	256	256	0.027	0.117	0.23x	12.985
120	1024	64	64	0.034	0.893	0.04x	12.784
120	1024	128	128	0.039	0.517	0.07x	12.784
120	1024	256	256	0.026	0.285	0.09x	12.784
120	4096	64	64	0.031	3.534	0.01x	12.608
120	4096	128	128	0.027	1.643	0.02x	12.608
120	4096	256	256	0.033	1.122	0.03x	12.608
440	8	64	8	0.117	0.076	1.54x	19.145
440	8	128	8	0.103	0.069	1.50x	19.145
440	8	256	8	0.077	0.055	1.39x	19.145
440	64	64	64	0.091	0.112	0.81x	5.821
440	64	128	64	0.081	0.093	0.87x	5.821
440	64	256	64	0.087	0.120	0.72x	5.821
440	512	64	64	0.078	0.568	0.14x	8.483
440	512	128	128	0.085	0.263	0.32x	8.483
440	512	256	256	0.075	0.228	0.33x	8.483
440	1024	64	64	0.091	1.043	0.09x	7.593
440	1024	128	128	0.083	0.533	0.16x	7.593
440	1024	256	256	0.075	0.312	0.24x	7.593
440	4096	64	64	0.075	4.004	0.02x	6.395
440	4096	128	128	0.080	2.255	0.04x	6.395
440	4096	256	256	0.097	1.199	0.08x	6.395
960	8	64	8	0.201	0.078	2.58x	30.489
960	8	128	8	0.153	0.090	1.70x	30.489
960	8	256	8	0.158	0.053	3.00x	30.489
960	64	64	64	0.159	0.179	0.89x	8.910
960	64	128	64	0.164	0.132	1.24x	8.910
960	64	256	64	0.174	0.117	1.48x	8.910
960	512	64	64	0.156	0.579	0.27x	6.091
960	512	128	128	0.149	0.338	0.44x	6.091
960	512	256	256	0.147	0.248	0.59x	6.091
960	1024	64	64	0.150	1.039	0.14x	5.397
960	1024	128	128	0.155	0.647	0.24x	5.397
960	1024	256	256	0.165	0.394	0.42x	5.397
960	4096	64	64	0.150	4.244	0.04x	4.598
960	4096	128	128	0.146	2.468	0.06x	4.598
960	4096	256	256	0.147	1.437	0.10x	4.598
5040	8	64	8	1.010	0.083	12.21x	21.737
5040	8	128	8	0.704	0.051	13.80x	21.737
5040	8	256	8	0.739	0.086	8.55x	21.737
5040	64	64	64	0.703	0.249	2.82x	9.197
5040	64	128	64	0.715	0.211	3.38x	9.197
5040	64	256	64	0.720	0.141	5.10x	9.197
5040	512	64	64	0.703	0.937	0.75x	3.581
5040	512	128	128	0.723	0.693	1.04x	3.581
5040	512	256	256	0.737	0.667	1.10x	3.581
5040	1024	64	64	0.718	1.458	0.49x	3.912
5040	1024	128	128	0.739	1.068	0.69x	3.912
5040	1024	256	256	0.753	0.952	0.79x	3.912
5040	4096	64	64	0.768	5.203	0.15x	2.023
5040	4096	128	128	0.718	3.272	0.22x	2.023
5040	4096	256	256	0.691	2.523	0.27x	2.023
10200	8	64	8	1.878	0.147	12.76x	23.893
10200	8	128	8	1.392	0.053	26.07x	23.893
10200	8	256	8	1.462	0.075	19.45x	23.893
10200	64	64	64	1.449	0.289	5.01x	7.635
10200	64	128	64	1.408	0.207	6.80x	7.635
10200	64	256	64	1.440	0.205	7.03x	7.635
10200	512	64	64	1.526	1.182	1.29x	4.252
10200	512	128	128	1.483	0.768	1.93x	4.252
10200	512	256	256	1.510	0.759	1.99x	4.252
10200	1024	64	64	1.484	1.485	1.00x	2.145
10200	1024	128	128	1.438	1.236	1.16x	2.145
10200	1024	256	256	1.492	1.300	1.15x	2.145
10200	4096	64	64	1.461	5.010	0.29x	1.738
10200	4096	128	128	1.398	4.305	0.32x	1.738
10200	4096	256	256	1.405	3.609	0.39x	1.738
50624	8	64	8	9.121	0.096	94.76x	40.820
50624	8	128	8	6.980	0.088	79.26x	40.820
50624	8	256	8	6.637	0.075	88.19x	40.820
50624	64	64	64	6.725	0.293	22.93x	21.174
50624	64	128	64	6.898	0.277	24.87x	21.174
50624	64	256	64	7.049	0.289	24.41x	21.174
50624	512	64	64	6.864	1.453	4.72x	5.468
50624	512	128	128	7.019	1.086	6.46x	5.468
50624	512	256	256	6.987	1.129	6.19x	5.468
50624	1024	64	64	6.646	2.210	3.01x	2.349
50624	1024	128	128	6.649	1.804	3.69x	2.349
50624	1024	256	256	6.691	2.135	3.13x	2.349
50624	4096	64	64	6.853	6.977	0.98x	1.172
50624	4096	128	128	6.759	5.822	1.16x	1.172
50624	4096	256	256	6.990	5.908	1.18x	1.172

Table 2: 30k 网格 (36470 cells): 端到端时间与同网格 S316 参考误差

方向数	样本数	请求 B	实际 B	SI total/s	RSI total/s	RSI/SI	error (%)
120	8	64	8	0.105	0.137	0.77x	23.443
120	8	128	8	0.077	0.126	0.61x	23.443
120	8	256	8	0.078	0.101	0.77x	23.443
120	64	64	64	0.084	0.107	0.78x	12.508
120	64	128	64	0.080	0.096	0.83x	12.508
120	64	256	64	0.083	0.114	0.73x	12.508
120	512	64	64	0.079	0.933	0.09x	11.663
120	512	128	128	0.079	0.582	0.14x	11.663
120	512	256	256	0.079	0.403	0.20x	11.663
120	1024	64	64	0.077	1.583	0.05x	11.157
120	1024	128	128	0.080	1.184	0.07x	11.157
120	1024	256	256	0.078	0.892	0.09x	11.157
120	4096	64	64	0.088	7.206	0.01x	11.239
120	4096	128	128	0.082	4.567	0.02x	11.239
120	4096	256	256	0.088	3.244	0.03x	11.239
440	8	64	8	0.343	0.171	2.00x	19.501
440	8	128	8	0.250	0.130	1.92x	19.501
440	8	256	8	0.235	0.204	1.15x	19.501

续表：30k 网格 (36470 cells)：端到端时间与同网格 S316 参考误差

方向数	样本数	请求 B	实际 B	SI total/s	RSI total/s	RSI/SI	error (%)
440	64	64	64	0.236	0.260	0.91x	5.167
440	64	128	64	0.234	0.205	1.14x	5.167
440	64	256	64	0.232	0.229	1.01x	5.167
440	512	64	64	0.237	1.219	0.19x	8.865
440	512	128	128	0.246	0.655	0.38x	8.865
440	512	256	256	0.268	0.560	0.48x	8.865
440	1024	64	64	0.237	2.237	0.11x	7.838
440	1024	128	128	0.236	1.382	0.17x	7.838
440	1024	256	256	0.250	1.009	0.25x	7.838
440	4096	64	64	0.248	8.447	0.03x	5.941
440	4096	128	128	0.250	5.280	0.05x	5.941
440	4096	256	256	0.237	3.682	0.06x	5.941
960	8	64	8	16.880	1.307	12.91x	29.438
960	8	128	8	16.281	1.227	13.27x	29.438
960	8	256	8	16.621	1.216	13.67x	29.438
960	64	64	64	16.401	3.566	4.60x	8.515
960	64	128	64	16.038	3.555	4.51x	8.515
960	64	256	64	16.303	3.537	4.61x	8.515
960	512	64	64	16.497	31.384	0.53x	5.972
960	512	128	128	17.575	28.706	0.61x	5.972
960	512	256	256	16.596	24.033	0.69x	5.972
960	1024	64	64	16.579	54.950	0.30x	5.301
960	1024	128	128	16.507	48.553	0.34x	5.301
960	1024	256	256	16.615	44.225	0.38x	5.301
960	4096	64	64	16.639	226.747	0.07x	4.116
960	4096	128	128	17.037	204.750	0.08x	4.116
960	4096	256	256	16.786	180.022	0.09x	4.116
5040	8	64	8	36.049	0.241	149.75x	18.626
5040	8	128	8	35.121	0.190	185.22x	18.626
5040	8	256	8	35.955	0.164	218.65x	18.626
5040	64	64	64	35.877	3.137	11.44x	7.850
5040	64	128	64	35.636	2.800	12.73x	7.850
5040	64	256	64	35.889	3.004	11.95x	7.850
5040	512	64	64	36.049	22.849	1.58x	3.764
5040	512	128	128	35.908	21.371	1.68x	3.764
5040	512	256	256	36.381	20.016	1.82x	3.764
5040	1024	64	64	37.680	29.969	1.26x	4.013
5040	1024	128	128	35.706	29.148	1.22x	4.013
5040	1024	256	256	35.536	25.849	1.37x	4.013
5040	4096	64	64	35.357	104.329	0.34x	1.945
5040	4096	128	128	34.532	100.030	0.35x	1.945
5040	4096	256	256	34.855	88.341	0.39x	1.945
10200	8	64	8	38.344	0.248	154.68x	21.487
10200	8	128	8	36.628	0.138	264.93x	21.487
10200	8	256	8	36.523	0.177	206.83x	21.487
10200	64	64	64	36.662	0.638	57.43x	6.297
10200	64	128	64	37.094	0.584	63.49x	6.297
10200	64	256	64	36.780	0.544	67.60x	6.297
10200	512	64	64	36.668	13.214	2.77x	3.290
10200	512	128	128	43.879	10.197	4.30x	3.290
10200	512	256	256	43.092	10.961	3.93x	3.290
10200	1024	64	64	39.247	16.816	2.33x	1.639
10200	1024	128	128	36.959	14.127	2.62x	1.639
10200	1024	256	256	36.320	13.688	2.65x	1.639
10200	4096	64	64	37.022	54.343	0.68x	1.602
10200	4096	128	128	36.028	49.800	0.72x	1.602
10200	4096	256	256	36.001	47.395	0.76x	1.602
50624	8	64	8	140.023	0.212	659.60x	35.121
50624	8	128	8	135.960	0.179	759.44x	35.121
50624	8	256	8	130.253	0.108	1201.17x	35.121
50624	64	64	64	129.774	2.792	46.49x	19.619
50624	64	128	64	129.525	2.687	48.21x	19.619
50624	64	256	64	129.269	2.750	47.00x	19.619
50624	512	64	64	161.923	12.237	13.23x	4.767
50624	512	128	128	166.438	10.768	15.46x	4.767
50624	512	256	256	169.810	9.654	17.59x	4.767
50624	1024	64	64	164.983	14.386	11.47x	2.395
50624	1024	128	128	155.774	15.516	10.04x	2.395
50624	1024	256	256	136.740	13.874	9.86x	2.395
50624	4096	64	64	154.865	48.708	3.18x	1.054
50624	4096	128	128	168.836	45.147	3.74x	1.054
50624	4096	256	256	145.068	46.633	3.11x	1.054

Table 3: 100k 网格 (94378 cells)：端到端时间与同网格 S316 参考误差；缺少 S224/4096/请求 B=256

方向数	样本数	请求 B	实际 B	SI total/s	RSI total/s	RSI/SI	error (%)
120	8	64	8	0.539	0.459	1.17x	30.574
120	8	128	8	0.205	0.217	0.95x	30.574
120	8	256	8	0.196	0.205	0.96x	30.574
120	64	64	64	0.188	0.227	0.83x	16.386
120	64	128	64	0.189	0.213	0.89x	16.386
120	64	256	64	0.187	0.214	0.87x	16.386
120	512	64	64	0.193	1.486	0.13x	14.481
120	512	128	128	0.184	1.254	0.15x	14.481
120	512	256	256	0.186	0.950	0.20x	14.481
120	1024	64	64	0.181	3.251	0.06x	14.130
120	1024	128	128	0.184	2.425	0.08x	14.130
120	1024	256	256	0.182	1.906	0.10x	14.130
120	4096	64	64	0.184	13.437	0.01x	13.984
120	4096	128	128	0.185	9.472	0.02x	13.984
120	4096	256	256	0.195	7.662	0.03x	13.984
440	8	64	8	1.446	0.553	2.62x	23.194
440	8	128	8	0.622	0.248	2.51x	23.194
440	8	256	8	0.610	0.365	1.67x	23.194
440	64	64	64	0.639	1.267	0.50x	6.777
440	64	128	64	0.629	0.423	1.49x	6.777
440	64	256	64	0.632	0.412	1.53x	6.777
440	512	64	64	0.591	2.060	0.29x	9.297
440	512	128	128	0.594	1.527	0.39x	9.297
440	512	256	256	0.625	1.230	0.51x	9.297
440	1024	64	64	0.597	4.563	0.13x	8.364
440	1024	128	128	0.693	3.246	0.21x	8.364
440	1024	256	256	0.689	2.683	0.26x	8.364
440	4096	64	64	0.672	17.722	0.04x	7.006
440	4096	128	128	0.626	10.877	0.06x	7.006
440	4096	256	256	0.598	8.772	0.07x	7.006
960	8	64	8	3.789	0.723	5.24x	33.702
960	8	128	8	1.438	0.298	4.82x	33.702
960	8	256	8	1.484	0.313	4.75x	33.702
960	64	64	64	1.434	2.861	0.50x	10.201
960	64	128	64	1.463	0.798	1.83x	10.201
960	64	256	64	1.665	0.807	2.06x	10.201

续表：100k 网格 (94378 cells)：端到端时间与同网格 S316 参考误差；缺少 S224/4096/请求 B=256

方向数	样本数	请求 B	实际 B	SI total/s	RSI total/s	RSI/SI	error (%)
960	512	64	64	1.560	3.693	0.42x	6.815
960	512	128	128	1.447	2.362	0.61x	6.815
960	512	256	256	1.433	1.916	0.75x	6.815
960	1024	64	64	1.419	5.662	0.25x	5.989
960	1024	128	128	1.432	4.240	0.34x	5.989
960	1024	256	256	1.383	3.250	0.43x	5.989
960	4096	64	64	1.406	21.263	0.07x	5.025
960	4096	128	128	1.271	14.328	0.09x	5.025
960	4096	256	256	1.247	10.128	0.12x	5.025
5040	8	64	8	17.449	0.662	26.35x	24.457
5040	8	128	8	6.325	0.217	29.15x	24.457
5040	8	256	8	5.955	0.293	20.33x	24.457
5040	64	64	64	5.873	3.966	1.48x	9.633
5040	64	128	64	5.982	1.465	4.08x	9.633
5040	64	256	64	6.056	1.556	3.89x	9.633
5040	512	64	64	6.201	12.680	0.49x	4.001
5040	512	128	128	6.116	4.837	1.26x	4.001
5040	512	256	256	6.303	4.960	1.27x	4.001
5040	1024	64	64	6.276	11.034	0.57x	4.237
5040	1024	128	128	6.093	7.925	0.77x	4.237
5040	1024	256	256	6.889	7.628	0.90x	4.237
5040	4096	64	64	6.090	30.928	0.20x	2.199
5040	4096	128	128	6.005	30.395	0.20x	2.199
5040	4096	256	256	7.348	25.028	0.29x	2.199
10200	8	64	8	14.849	0.775	19.15x	26.621
10200	8	128	8	13.038	0.256	50.97x	26.621
10200	8	256	8	13.012	0.283	45.98x	26.621
10200	64	64	64	12.515	1.188	10.54x	8.436
10200	64	128	64	12.032	1.290	9.33x	8.436
10200	64	256	64	11.438	1.257	9.10x	8.436
10200	512	64	64	11.428	5.464	2.09x	4.473
10200	512	128	128	11.639	5.070	2.30x	4.473
10200	512	256	256	12.729	7.408	1.72x	4.473
10200	1024	64	64	14.962	11.548	1.30x	2.231
10200	1024	128	128	13.264	11.419	1.16x	2.231
10200	1024	256	256	13.074	9.155	1.43x	2.231
10200	4096	64	64	13.316	34.837	0.38x	1.841
10200	4096	128	128	12.460	34.334	0.36x	1.841
10200	4096	256	256	13.590	30.248	0.45x	1.841
50624	8	64	8	134.551	0.812	165.61x	43.011
50624	8	128	8	134.012	0.263	510.40x	43.011
50624	8	256	8	127.902	0.252	507.72x	43.011
50624	64	64	64	168.503	6.394	26.35x	21.374
50624	64	128	64	144.489	1.439	100.44x	21.374
50624	64	256	64	136.794	1.066	128.31x	21.374
50624	512	64	64	136.577	6.017	22.70x	5.573
50624	512	128	128	139.865	6.334	22.08x	5.573
50624	512	256	256	141.051	8.060	17.50x	5.573
50624	1024	64	64	133.688	10.468	12.77x	2.562
50624	1024	128	128	139.067	12.018	11.57x	2.562
50624	1024	256	256	143.106	13.725	10.43x	2.562
50624	4096	64	64	138.122	44.853	3.08x	1.252
50624	4096	128	128	142.833	45.893	3.11x	1.252

Table 4: 10k 网格 (9187 cells)：SI/RSI 并行效率

方向数	样本数	请求 B	实际 B	SI sweep/s	E_{SI}	RSI sweep/s	E_{RSI}	RSI/SI	RSI plan/s
120	8	64	8	0.010	1498.6	0.035	50.1	0.03x	0.018
120	8	128	8	0.011	1459.5	0.037	47.3	0.03x	0.007
120	8	256	8	0.011	1414.9	0.055	32.2	0.02x	0.007
120	64	64	64	0.010	1488.9	0.035	403.8	0.27x	0.003
120	64	128	64	0.011	1436.2	0.038	373.7	0.26x	0.004
120	64	256	64	0.010	1498.6	0.037	385.1	0.26x	0.004
120	512	64	64	0.011	1375.0	0.342	330.1	0.24x	0.019
120	512	128	128	0.010	1513.4	0.237	476.1	0.31x	0.012
120	512	256	256	0.011	1459.7	0.097	1159.0	0.79x	0.006
120	1024	64	64	0.013	1215.2	0.775	291.2	0.24x	0.041
120	1024	128	128	0.011	1456.4	0.450	501.8	0.34x	0.022
120	1024	256	256	0.011	1454.8	0.246	917.2	0.63x	0.010
120	4096	64	64	0.011	1467.6	3.092	292.1	0.20x	0.154
120	4096	128	128	0.011	1438.2	1.429	632.1	0.44x	0.071
120	4096	256	256	0.010	1499.4	0.980	921.5	0.61x	0.039
440	8	64	8	0.047	1200.6	0.042	42.3	0.04x	0.029
440	8	128	8	0.046	1223.9	0.047	37.8	0.03x	0.017
440	8	256	8	0.042	1340.2	0.039	44.9	0.03x	0.011
440	64	64	64	0.048	1177.8	0.043	326.3	0.28x	0.063
440	64	128	64	0.045	1256.4	0.049	290.3	0.23x	0.037
440	64	256	64	0.048	1182.8	0.056	251.8	0.21x	0.055
440	512	64	64	0.042	1347.1	0.447	252.3	0.19x	0.076
440	512	128	128	0.047	1215.5	0.186	605.9	0.50x	0.054
440	512	256	256	0.043	1321.1	0.170	665.1	0.50x	0.039
440	1024	64	64	0.049	1157.1	0.837	269.7	0.23x	0.123
440	1024	128	128	0.047	1204.9	0.413	546.8	0.45x	0.077
440	1024	256	256	0.042	1339.9	0.221	1022.1	0.76x	0.063
440	4096	64	64	0.042	1353.1	3.278	275.5	0.20x	0.406
440	4096	128	128	0.043	1322.2	1.837	491.7	0.37x	0.232
440	4096	256	256	0.050	1137.5	0.963	938.1	0.82x	0.124
960	8	64	8	0.087	1420.4	0.047	37.3	0.03x	0.025
960	8	128	8	0.087	1412.5	0.067	26.5	0.02x	0.017
960	8	256	8	0.091	1361.7	0.037	48.0	0.04x	0.011
960	64	64	64	0.089	1391.5	0.056	250.9	0.18x	0.115
960	64	128	64	0.095	1296.6	0.062	226.8	0.17x	0.061
960	64	256	64	0.097	1277.1	0.051	277.9	0.22x	0.059
960	512	64	64	0.091	1352.2	0.393	287.5	0.21x	0.138
960	512	128	128	0.085	1449.1	0.201	561.3	0.39x	0.110
960	512	256	256	0.086	1430.5	0.130	870.5	0.61x	0.096
960	1024	64	64	0.086	1442.1	0.752	300.3	0.21x	0.204
960	1024	128	128	0.091	1357.6	0.410	551.2	0.41x	0.179
960	1024	256	256	0.093	1332.1	0.227	995.1	0.75x	0.132
960	4096	64	64	0.087	1417.0	3.206	281.7	0.20x	0.695
960	4096	128	128	0.085	1444.6	1.749	516.4	0.36x	0.477
960	4096	256	256	0.084	1475.8	0.994	908.6	0.62x	0.299
5040	8	64	8	0.418	1552.0	0.043	41.4	0.03x	0.035
5040	8	128	8	0.446	1454.1	0.035	50.6	0.03x	0.012
5040	8	256	8	0.457	1417.7	0.055	31.9	0.02x	0.025
5040	64	64	64	0.427	1519.4	0.057	249.1	0.16x	0.183
5040	64	128	64	0.454	1428.4	0.038	370.5	0.26x	0.165
5040	64	256	64	0.450	1440.4	0.035	398.7	0.28x	0.098
5040	512	64	64	0.432	1502.0	0.471	239.7	0.16x	0.395
5040	512	128	128	0.445	1455.5	0.189	598.4	0.41x	0.461
5040	512	256	256	0.437	1482.5	0.131	859.2	0.58x	0.503
5040	1024	64	64	0.435	1489.5	0.936	241.2	0.16x	0.387

续表：10k 网格 (9187 cells): SI/RSI 并行效率

方向数	样本数	请求 B	实际 B	SI sweep/s	E_{SI}	RSI sweep/s	E_{RSI}	RSI/SI	RSI plan/s
5040	1024	128	128	0.455	1423.2	0.423	533.5	0.37x	0.564
5040	1024	256	256	0.453	1431.2	0.294	767.6	0.54x	0.596
5040	4096	64	64	0.452	1435.0	3.406	265.1	0.18x	1.279
5040	4096	128	128	0.434	1494.4	1.659	544.4	0.36x	1.278
5040	4096	256	256	0.426	1522.7	1.082	835.0	0.55x	1.198
10200	8	64	8	0.867	1512.4	0.098	18.0	0.01x	0.041
10200	8	128	8	0.864	1518.8	0.035	50.8	0.03x	0.014
10200	8	256	8	0.867	1512.4	0.047	37.9	0.03x	0.022
10200	64	64	64	0.882	1487.2	0.046	306.8	0.21x	0.232
10200	64	128	64	0.872	1504.2	0.040	353.0	0.23x	0.158
10200	64	256	64	0.876	1498.4	0.042	332.2	0.22x	0.152
10200	512	64	64	0.898	1461.5	0.581	194.3	0.13x	0.505
10200	512	128	128	0.883	1485.0	0.181	625.1	0.42x	0.533
10200	512	256	256	0.909	1442.6	0.107	1056.8	0.73x	0.608
10200	1024	64	64	0.887	1479.1	0.884	255.3	0.17x	0.445
10200	1024	128	128	0.882	1486.7	0.458	493.2	0.33x	0.669
10200	1024	256	256	0.916	1432.2	0.292	773.3	0.54x	0.917
10200	4096	64	64	0.882	1487.3	3.034	297.7	0.20x	1.410
10200	4096	128	128	0.862	1522.0	2.045	441.6	0.29x	1.831
10200	4096	256	256	0.871	1505.8	1.049	861.3	0.57x	2.228
50624	8	64	8	4.312	1510.1	0.052	33.9	0.02x	0.037
50624	8	128	8	4.347	1497.8	0.059	30.0	0.02x	0.022
50624	8	256	8	4.199	1550.5	0.051	34.3	0.02x	0.017
50624	64	64	64	4.364	1492.0	0.036	388.0	0.26x	0.237
50624	64	128	64	4.421	1472.9	0.037	378.0	0.26x	0.221
50624	64	256	64	4.249	1532.3	0.047	299.6	0.20x	0.223
50624	512	64	64	4.266	1526.3	0.443	255.1	0.17x	0.866
50624	512	128	128	4.246	1533.5	0.179	629.7	0.41x	0.786
50624	512	256	256	4.303	1513.3	0.117	967.7	0.64x	0.892
50624	1024	64	64	4.229	1539.5	0.740	305.0	0.20x	1.163
50624	1024	128	128	4.210	1546.6	0.408	553.0	0.36x	1.145
50624	1024	256	256	4.241	1535.3	0.300	753.6	0.49x	1.582
50624	4096	64	64	4.240	1535.7	3.218	280.6	0.18x	2.555
50624	4096	128	128	4.335	1502.1	1.772	509.8	0.34x	3.019
50624	4096	256	256	4.333	1502.6	1.091	827.7	0.55x	3.831

Table 5: 30k 网格 (36470 cells): SI/RSI 并行效率

方向数	样本数	请求 B	实际 B	SI sweep/s	E_{SI}	RSI sweep/s	E_{RSI}	RSI/SI	RSI plan/s
120	8	64	8	0.034	1798.5	0.072	97.8	0.05x	0.059
120	8	128	8	0.033	1877.3	0.095	73.5	0.04x	0.024
120	8	256	8	0.033	1878.5	0.068	103.3	0.05x	0.027
120	64	64	64	0.033	1835.3	0.084	665.7	0.36x	0.016
120	64	128	64	0.033	1834.0	0.075	748.7	0.41x	0.015
120	64	256	64	0.034	1813.6	0.093	600.7	0.33x	0.014
120	512	64	64	0.034	1776.4	0.822	545.4	0.31x	0.059
120	512	128	128	0.034	1820.2	0.509	880.6	0.48x	0.036
120	512	256	256	0.033	1852.4	0.350	1281.7	0.69x	0.021
120	1024	64	64	0.033	1884.8	1.376	651.4	0.35x	0.108
120	1024	128	128	0.033	1862.6	1.038	863.7	0.46x	0.060
120	1024	256	256	0.033	1863.4	0.794	1128.7	0.61x	0.033
120	4096	64	64	0.037	1634.6	6.341	565.4	0.35x	0.415
120	4096	128	128	0.035	1753.1	4.013	893.3	0.51x	0.219
120	4096	256	256	0.039	1567.5	2.870	1249.2	0.80x	0.112
440	8	64	8	0.128	1750.9	0.072	97.6	0.06x	0.091
440	8	128	8	0.130	1726.9	0.085	82.0	0.05x	0.036
440	8	256	8	0.123	1824.3	0.156	44.8	0.02x	0.037
440	64	64	64	0.122	1848.6	0.128	439.0	0.24x	0.120
440	64	128	64	0.122	1834.8	0.090	624.1	0.34x	0.105
440	64	256	64	0.123	1832.2	0.112	500.2	0.27x	0.105
440	512	64	64	0.125	1792.4	0.885	506.6	0.28x	0.249
440	512	128	128	0.128	1761.9	0.433	1034.4	0.59x	0.174
440	512	256	256	0.143	1575.2	0.388	1154.6	0.73x	0.132
440	1024	64	64	0.124	1814.1	1.669	537.1	0.30x	0.410
440	1024	128	128	0.124	1808.1	1.025	874.4	0.48x	0.254
440	1024	256	256	0.126	1780.9	0.755	1187.7	0.67x	0.177
440	4096	64	64	0.131	1709.8	6.357	564.0	0.33x	1.417
440	4096	128	128	0.130	1732.0	4.072	880.4	0.51x	0.784
440	4096	256	256	0.125	1801.6	3.025	1185.3	0.66x	0.417
960	8	64	8	15.928	30.8	1.168	6.0	0.19x	0.104
960	8	128	8	15.800	31.0	1.151	6.1	0.20x	0.042
960	8	256	8	16.013	30.6	1.140	6.1	0.20x	0.041
960	64	64	64	15.801	31.0	3.275	17.1	0.55x	0.198
960	64	128	64	15.448	31.7	3.286	17.0	0.54x	0.176
960	64	256	64	15.715	31.2	3.267	17.1	0.55x	0.176
960	512	64	64	15.901	30.8	30.127	14.9	0.48x	0.425
960	512	128	128	16.937	28.9	27.614	16.2	0.56x	0.346
960	512	256	256	15.982	30.7	23.151	19.4	0.63x	0.262
960	1024	64	64	15.964	30.7	52.771	17.0	0.55x	0.707
960	1024	128	128	15.901	30.8	46.763	19.2	0.62x	0.511
960	1024	256	256	15.988	30.7	42.703	21.0	0.68x	0.350
960	4096	64	64	16.004	30.6	218.280	16.4	0.54x	2.393
960	4096	128	128	16.410	29.9	197.789	18.1	0.61x	1.523
960	4096	256	256	16.148	30.4	174.429	20.6	0.68x	0.881
5040	8	64	8	33.160	77.6	0.067	104.5	1.35x	0.167
5040	8	128	8	33.000	78.0	0.074	95.0	1.22x	0.108
5040	8	256	8	33.846	76.0	0.074	94.4	1.24x	0.082
5040	64	64	64	33.892	75.9	2.274	24.6	0.32x	0.792
5040	64	128	64	33.713	76.3	2.263	24.8	0.32x	0.462
5040	64	256	64	33.936	75.8	2.384	23.5	0.31x	0.537
5040	512	64	64	33.954	75.8	21.242	21.1	0.28x	1.042
5040	512	128	128	34.026	75.6	19.522	23.0	0.30x	1.207
5040	512	256	256	33.636	76.5	17.785	25.2	0.33x	1.682
5040	1024	64	64	34.898	73.7	27.565	32.5	0.44x	1.478
5040	1024	128	128	33.400	77.0	26.677	33.6	0.44x	1.598
5040	1024	256	256	33.641	76.5	23.540	38.1	0.50x	1.559
5040	4096	64	64	33.072	77.8	96.050	37.3	0.48x	4.907
5040	4096	128	128	32.522	79.1	92.597	38.7	0.49x	4.284
5040	4096	256	256	33.038	77.9	82.145	43.6	0.56x	3.449
10200	8	64	8	34.128	152.6	0.107	65.4	0.43x	0.132
10200	8	128	8	33.941	153.4	0.084	83.8	0.55x	0.043
10200	8	256	8	33.995	153.2	0.118	59.3	0.39x	0.050
10200	64	64	64	34.120	152.6	0.169	330.9	2.17x	0.449
10200	64	128	64	34.509	150.9	0.144	388.9	2.58x	0.420
10200	64	256	64	34.152	152.5	0.107	521.7	3.42x	0.418
10200	512	64	64	34.076	152.8	8.892	50.4	0.33x	3.938
10200	512	128	128	39.090	133.2	7.551	59.3	0.45x	2.305
10200	512	256	256	39.124	133.1	6.985	64.2	0.48x	3.740
10200	1024	64	64	36.046	144.5	14.237	63.0	0.44x	1.942
10200	1024	128	128	34.395	151.4	11.835	75.7	0.50x	1.734
10200	1024	256	256	33.747	154.3	11.066	81.0	0.52x	2.155
10200	4096	64	64	33.471	155.6	46.471	77.1	0.50x	5.605

续表：30k 网格 (36470 cells)：SI/RSI 并行效率

方向数	样本数	请求 B	实际 B	SI sweep/s	E_{SI}	RSI sweep/s	E_{RSI}	RSI/SI	RSI plan/s
10200	4096	128	128	33.413	155.9	42.642	84.1	0.54x	5.139
10200	4096	256	256	33.398	155.9	41.080	87.3	0.56x	4.471
50624	8	64	8	121.100	213.4	0.086	81.5	0.38x	0.117
50624	8	128	8	119.657	216.0	0.068	102.7	0.48x	0.101
50624	8	256	8	119.683	216.0	0.066	105.8	0.49x	0.034
50624	64	64	64	119.940	215.5	2.229	25.1	0.12x	0.479
50624	64	128	64	119.985	215.4	2.126	26.3	0.12x	0.475
50624	64	256	64	119.686	216.0	2.172	25.8	0.12x	0.492
50624	512	64	64	151.092	171.1	9.999	44.8	0.26x	1.690
50624	512	128	128	154.407	167.4	8.189	54.7	0.33x	2.159
50624	512	256	256	155.417	166.3	7.165	62.5	0.38x	2.081
50624	1024	64	64	150.644	171.6	10.847	82.6	0.48x	2.664
50624	1024	128	128	140.629	183.8	11.602	77.3	0.42x	3.053
50624	1024	256	256	123.319	209.6	9.394	95.4	0.46x	3.803
50624	4096	64	64	139.898	184.8	38.646	92.8	0.50x	7.171
50624	4096	128	128	151.190	171.0	34.846	102.9	0.60x	7.514
50624	4096	256	256	129.098	200.2	35.584	100.8	0.50x	8.489

Table 6: 100k 网格 (94378 cells)：SI/RSI 并行效率；缺少 S224/4096/请求 B=256

方向数	样本数	请求 B	实际 B	SI sweep/s	E_{SI}	RSI sweep/s	E_{RSI}	RSI/SI	RSI plan/s
120	8	64	8	0.086	1845.5	0.148	122.2	0.07x	0.297
120	8	128	8	0.092	1728.9	0.144	126.2	0.07x	0.063
120	8	256	8	0.085	1857.0	0.131	138.1	0.07x	0.064
120	64	64	64	0.086	1838.1	0.173	839.7	0.46x	0.035
120	64	128	64	0.085	1858.2	0.159	910.9	0.49x	0.035
120	64	256	64	0.085	1870.9	0.162	894.9	0.48x	0.035
120	512	64	64	0.086	1845.2	1.235	939.1	0.51x	0.138
120	512	128	128	0.084	1882.5	1.081	1073.2	0.57x	0.078
120	512	256	256	0.085	1875.8	0.826	1403.5	0.75x	0.049
120	1024	64	64	0.084	1887.2	2.780	834.4	0.44x	0.256
120	1024	128	128	0.085	1872.9	2.118	1094.9	0.58x	0.138
120	1024	256	256	0.084	1896.1	1.682	1378.7	0.73x	0.078
120	4096	64	64	0.085	1873.0	11.600	799.8	0.43x	0.980
120	4096	128	128	0.085	1873.1	8.351	1111.0	0.59x	0.496
120	4096	256	256	0.092	1714.2	6.849	1354.5	0.79x	0.259
440	8	64	8	0.316	1837.6	0.147	122.9	0.07x	0.390
440	8	128	8	0.325	1790.6	0.138	131.7	0.07x	0.098
440	8	256	8	0.318	1828.5	0.252	72.0	0.04x	0.097
440	64	64	64	0.339	1714.2	0.160	904.1	0.53x	1.086
440	64	128	64	0.341	1706.6	0.168	862.0	0.51x	0.234
440	64	256	64	0.358	1625.5	0.151	959.7	0.59x	0.239
440	512	64	64	0.318	1829.4	1.247	930.2	0.51x	0.666
440	512	128	128	0.317	1833.4	1.021	1136.3	0.62x	0.396
440	512	256	256	0.323	1798.6	0.850	1364.1	0.76x	0.289
440	1024	64	64	0.321	1811.3	3.134	740.0	0.41x	1.028
440	1024	128	128	0.386	1507.3	2.282	1016.5	0.67x	0.715
440	1024	256	256	0.389	1495.3	2.051	1131.0	0.76x	0.433
440	4096	64	64	0.377	1543.5	12.316	753.3	0.49x	3.945
440	4096	128	128	0.334	1739.8	7.908	1173.2	0.67x	2.107
440	4096	256	256	0.323	1797.3	7.059	1314.4	0.73x	1.025
960	8	64	8	0.805	1575.2	0.157	115.1	0.07x	0.550
960	8	128	8	0.809	1568.7	0.158	114.5	0.07x	0.125
960	8	256	8	0.822	1542.2	0.171	105.9	0.07x	0.126
960	64	64	64	0.819	1548.6	0.194	748.1	0.48x	2.630
960	64	128	64	0.823	1541.4	0.185	781.8	0.51x	0.580
960	64	256	64	0.814	1558.6	0.161	903.0	0.58x	0.620
960	512	64	64	0.814	1557.6	1.517	764.6	0.49x	1.854
960	512	128	128	0.823	1540.9	1.087	1066.7	0.69x	1.107
960	512	256	256	0.816	1555.1	1.033	1123.1	0.72x	0.754
960	1024	64	64	0.810	1565.2	2.840	816.6	0.52x	2.322
960	1024	128	128	0.804	1577.9	2.131	1088.3	0.69x	1.804
960	1024	256	256	0.809	1568.0	2.031	1141.8	0.73x	0.985
960	4096	64	64	0.819	1548.6	11.486	807.8	0.52x	7.736
960	4096	128	128	0.734	1728.2	7.703	1204.4	0.70x	5.540
960	4096	256	256	0.709	1788.4	7.067	1312.9	0.73x	2.247
5040	8	64	8	3.656	1821.5	0.120	150.6	0.08x	0.532
5040	8	128	8	3.675	1812.0	0.130	139.9	0.08x	0.077
5040	8	256	8	3.613	1843.3	0.203	89.3	0.05x	0.077
5040	64	64	64	3.597	1851.6	0.189	768.8	0.42x	3.740
5040	64	128	64	3.608	1845.6	0.222	653.6	0.35x	1.208
5040	64	256	64	3.658	1820.6	0.188	769.5	0.42x	1.336
5040	512	64	64	3.675	1812.1	1.354	856.7	0.47x	10.947
5040	512	128	128	3.610	1844.6	0.934	1242.3	0.67x	3.673
5040	512	256	256	3.960	1681.6	0.919	1262.4	0.75x	3.805
5040	1024	64	64	3.679	1810.2	2.893	801.8	0.44x	7.350
5040	1024	128	128	3.691	1804.1	2.112	1098.4	0.61x	5.125
5040	1024	256	256	4.253	1566.0	1.925	1204.7	0.77x	5.215
5040	4096	64	64	3.633	1833.1	11.088	836.7	0.46x	17.224
5040	4096	128	128	3.611	1844.0	8.933	1038.6	0.56x	19.105
5040	4096	256	256	4.266	1561.1	8.244	1125.4	0.72x	14.579
10200	8	64	8	8.400	1604.4	0.143	126.4	0.08x	0.619
10200	8	128	8	8.636	1560.6	0.150	121.1	0.08x	0.093
10200	8	256	8	8.631	1561.4	0.158	114.8	0.07x	0.101
10200	64	64	64	8.037	1676.8	0.142	1023.8	0.61x	1.007
10200	64	128	64	7.516	1793.2	0.183	793.6	0.44x	1.074
10200	64	256	64	7.444	1810.4	0.197	737.3	0.41x	1.028
10200	512	64	64	7.337	1836.8	1.290	898.7	0.49x	3.765
10200	512	128	128	7.496	1798.0	0.954	1216.0	0.68x	3.798
10200	512	256	256	7.302	1845.8	0.936	1238.6	0.67x	6.227
10200	1024	64	64	8.740	1542.1	2.818	823.1	0.53x	8.102
10200	1024	128	128	8.330	1617.9	1.923	1206.1	0.75x	8.958
10200	1024	256	256	7.309	1844.0	1.753	1322.9	0.72x	6.913
10200	4096	64	64	7.333	1838.0	11.155	831.7	0.45x	21.271
10200	4096	128	128	7.418	1816.8	8.066	1150.2	0.63x	23.987
10200	4096	256	256	7.391	1823.4	6.952	1334.5	0.73x	21.259
50624	8	64	8	39.909	1676.0	0.142	128.0	0.08x	0.656
50624	8	128	8	40.256	1661.6	0.143	127.0	0.08x	0.107
50624	8	256	8	37.445	1786.3	0.142	127.3	0.07x	0.096
50624	64	64	64	51.375	1302.0	0.174	834.3	0.64x	6.143
50624	64	128	64	42.930	1558.1	0.275	527.8	0.34x	1.108
50624	64	256	64	38.965	1716.7	0.154	941.2	0.55x	0.866
50624	512	64	64	37.517	1782.9	1.451	799.4	0.45x	4.026
50624	512	128	128	39.635	1687.6	1.085	1069.0	0.63x	4.799
50624	512	256	256	41.169	1624.8	1.019	1137.9	0.70x	6.587
50624	1024	64	64	38.720	1727.5	2.580	898.9	0.52x	7.079
50624	1024	128	128	42.669	1567.6	2.140	1083.7	0.69x	9.039
50624	1024	256	256	42.633	1569.0	2.050	1131.4	0.72x	10.793
50624	4096	64	64	39.466	1694.9	11.997	773.3	0.46x	27.635
50624	4096	128	128	42.007	1592.3	8.642	1073.6	0.67x	33.531

影响因素与低误差最优组合 端到端加速的首要影响因素是角度方向数：SI 必须计算全部 N_Ω 个方向，而 RSI 每步只推进一条样本链。固定 30k 网格、4,096 样本和请求 $B = 128$ 时， S_{70} (5,040 directions)、 S_{100} (10,200 directions) 和 S_{224} (50,624 directions) 的端到端加速比分别为 $0.345\times$ 、 $0.723\times$ 和 $3.740\times$ ；相应 error 为 1.945%、1.602% 和 1.054%。样本数是精度-时间权衡的主导因素：同一 $30k/S_{224}/B = 128$ 下，8、64、512、1,024、4,096 样本的 error 从 35.121%、19.619%、4.767%、2.395% 降至 1.054%，但端到端加速也从 $759.439\times$ 、 $48.210\times$ 、 $15.456\times$ 、 $10.040\times$ 降至 $3.740\times$ 。batch 则主要影响 RSI 的 kernel 吞吐和 plan 调度；在该低误差点， $B = 64/128/256$ 的端到端加速分别为 $3.179\times$ 、 $3.740\times$ 、 $3.111\times$ ，故 $B = 128$ 为本机的甜点。空间网格会进一步改变 plan/cache 成本；例如同一 $S_{224}/4,096/B = 128$ 在 100k 上的单 sweep 吞吐比为 $0.674\times$ ，高于 30k 的 $0.602\times$ ，但 100k 的 RSI plan 时间为 33.531 s，端到端加速降为 $3.112\times$ 。

在 error 不超过 2% 的 23 组中， $30k/S_{224}/4,096/B = 128$ 的端到端加速最高，且 error 在全部 269 组中最低。该 case 的 SI 收敛轮数为 14，RSI-tail 使用 $14 + 10 = 24$ 步，因此 SI 需要 $50,624 \times 14 = 708,736$ 次 directional cell-sweep/cell，RSI 需要 $4,096 \times 24 = 98,304$ 次 sample-sweep/cell，工作量减少 $7.21\times$ 。其 sweep kernel 统计如下；block 和 logical thread 均为整个 SI fine 或 RSI 过程的累计 launch 几何，不表示硬件实际活跃 lane 数。

路径	launches	blocks	logical threads	threads/block	E (Mcell-sweeps/s)	$E_{\text{RSI}}/E_{\text{SI}}$
SI	7,602	710,500	181,637,120	256	170.96	-
RSI	209,666	80,314,144	20,560,154,624	256	102.88	$0.602\times$

因此该 case 的 sweep 阶段实际加速为 $151.190/34.846 = 4.338\times$ ；RSI 的单位 sweep 吞吐只有 SI 的 $0.602\times$ ，端到端的 $3.740\times$ 优势来自所需方向/样本 sweep 数显著减少，而不是 RSI 单个 kernel 的并行吞吐更高。

Nsight Systems/Compute 采样 对同一 case 的完整 Nsight Systems trace 共记录 227,160 次 cudaLaunchKernel。GPU kernel 时间的 62.9% 位于 sweepSamplesKernel，36.5% 位于 sweepLevelFusedKernel。主机端 kernel-launch API 仅占 0.3% 时间；其余等待主要对应 GPU 同步，而非 host launch 调用。该 case 的主问题因此是 GPU sweep 执行。trace 下的绝对时间受 profiling 记录开销影响，以下仅使用其 kernel 占比和实例数作归因。

Nsight Systems kernel	GPU time share	instances	total GPU time/s
sweepSamplesKernel	62.9%	560	131.323
sweepLevelFusedKernel	36.5%	1,281	76.209

Nsight Compute 对两个主导 kernel 各采一个代表性 launch；这些硬件计数用于衡量实际活跃 warp、缓存层级和 launch 尺度，不能由累计 block 数替代。sweepSamplesKernel 的采样 grid 只有 1 个 block，表明 cyclic/fallback 路径存在严重的单 block 低占用；sweepLevelFusedKernel 的 grid 为 128 个 block，仍只有 0.37 waves/SM，且其 L1 命中率低、L2 命中率高，符合 L2/访存和 wavefront 负载受限的特征。

NCU sampled kernel	grid	block	achieved/theoretical occ.	waves/SM	L1 hit	L2 hit	DRAM throughput
sweepSamplesKernel	1	128	2.08% / 83.33%	0.00	15.33%	10.34%	1.32%
sweepLevelFusedKernel	128	256	30.35% / 83.33%	0.37	9.48%	97.14%	7.16%

两次 NCU 采样还分别报告约 39% 和 75% 的 global-memory sectors 为 excessive/uncoalesced。优化优先级因此为：先减少 cyclic sweepSamplesKernel 的低 grid 规模或将其与其他样本合并，再改善 fused wavefront 的全局访问合并度和 level 内负载均衡；单纯增加 block 数不能解决这两类瓶颈。

数据、含义与结论汇总 下表将本节中容易混淆的计时、launch 几何和硬件计数统一解释。除 NCU 的两个代表性 launch 外，其余数据均来自 $30k/S_{224}/4,096/B = 128$ 的完整运行；NCU 单次采样只用于定位瓶颈，不能外推为每个 kernel 实例的精确平均值。

指标	数据	含义	结论
低误差最优点	$\text{error} = 1.054\%$ ；端到端加速 $= 3.740\times$	在 269 组中 error 最低，且在 $\text{error} \leq 2\%$ 的 23 组中端到端加速最高。	$30k/S_{224}/4,096/B = 128$ 是当前精度约束下的推荐配置。
工作量与单位吞吐	SI/RSI sweep 工作量 $= 708,736/98,304 = 7.21\times$ ； $E_{\text{RSI}}/E_{\text{SI}} = 0.602\times$	RSI 每个等价 cell-sweep 的吞吐低于 SI；但待执行的样本 sweep 数少得多。	$3.740\times$ 加速主要来自工作量减少，不表示 RSI 单位计算更快。
累计 launch 几何	SI：7,602 launches、710,500 blocks；RSI：209,666 launches、80,314,144 blocks；均为 256 threads/block	这是完整计算中各 kernel grid 的累加，包含方向、level、样本 batch 和 cyclic 回退的不同分解。	RSI block 总数大于 SI 并不构成比较不公平；它不能衡量实际 SM 活跃线程或硬件利用率。
主导 GPU 时间	sweepSamplesKernel：62.9%、560 instances；sweepLevelFusedKernel：36.5%、1,281 instances；host launch API：0.3%	时间主要花在 GPU sweep，本机 host 发射开销不是主导项。	优化应优先针对两个 sweep kernel 的并行度和访存，而非减少 host API 调用。
cyclic/fallback 利用率	sweepSamplesKernel: grid = 1、block = 128、achieved occupancy = 2.08%（理论 83.33%）	该代表性 launch 只有一个 block，远不足以覆盖 GPU 的 SM。	优先合并或批处理 cyclic/fallback 样本，减少单 block launch；仅增大每 block 线程数无效。
fused wavefront 利用率与缓存	sweepLevelFusedKernel: grid = 128、block = 256、occupancy = 30.35%、waves/SM = 0.37、L1/L2 hit = 9.48%/97.14%、DRAM = 7.16% peak、uncoalesced $\approx 75\%$	wavefront 已有一定并行度，但 level 宽度不足、L1 复用低且全局访问合并度差；高 L2 hit 不等于高 DRAM 带宽利用。	下一步应改善 level 内负载均衡和数据布局/访问合并度，而不是依据累计 block 数继续加大 grid。

9. 关键可调参数

环境变量	作用
RSI_CUDA_STREAMING_PLAN_THRESHOLD_MB	控制何时从全量 plan 切换到流式 sweep plan。
RSI_CUDA_SI_PLAN_CHUNK	SI 每个 plan 方向块的方向数，默认 128。
RSI_CUDA_SI_DEVICE_PLAN_CACHE_MB	SI 设备端 plan 常驻显存预算。
RSI_CUDA_PLAN_HOST_CACHE_MB	主机端 LRU plan 缓存预算。
RSI_SWEEP_PLAN_CACHE_DIR	磁盘 plan 缓存目录。
RSI_CUDA_SI_WAVEFRONT	开关 SI 无环波前并行扫描。
RSI_CUDA_SI_TILED_WAVEFRONT	开关 SI 依赖层分块并行。
RSI_CUDA_RSI_WAVEFRONT	开关 RSI 无环波前并行扫描。
RSI_CUDA_RSI_TILED_WAVEFRONT	开关 RSI 依赖层分块并行。
RSI_CUDA_RSI_FUSED_WAVEFRONT	开关 RSI fused wavefront；大网格默认开启。
RSI_CUDA_RSI_MAX_SAMPLES_PER_BATCH	控制 streaming RSI 每个 batch 的样本数，默认 128；host memory 紧张时可调小。
RSI_CUDA_RSI_PLAN_PREFETCH	是否在 GPU sweep 当前 RSI batch 时预取下一 batch 的 host plan，默认开启。
RSI_CUDA_RSI_ADAPTIVE_WAVEFRONT	是否启用 fused/tiled 自适应实验路径，默认关闭。
RSI_CUDA_RSI_ADAPTIVE_TILE_THRESHOLD	自适应路径切回 tiled wavefront 的 level tile 阈值。
RSI_CUDA_RSI_GRAPH_WAVEFRONT	是否对 streaming RSI level-by-level 路径做 CUDA Graph 捕获实验，默认关闭。
RSI_CUDA_RSI_FACE_LANE_WAVEFRONT	是否启用 face-lane wavefront 实验 kernel，默认关闭。
RSI_CUDA_FIXED_PLAN_CHUNK	RSI 固定方向缓存的方向粒度，默认 1。
RSI_CUDA_FIXED_PLAN_PARALLEL_LOAD	是否并行加载固定方向 plan 缓存。
RSI_CUDA_RSI_PLAN_PREWARM	是否在 RSI 前预热固定方向缓存。
RSI_CUDA_FIXED_PLAN_PREWARM_WORKERS	预热固定方向缓存的 CPU worker 数。
RSI_CUDA_FIXED_PLAN_MISS_DEDUP	是否对固定方向缓存 miss 去重。

10. 当前优化逻辑总结

当前实现的核心原则是：

1. SI 用角度并行和波前并行，把“方向间独立”和“同一依赖层内单元独立”都利用起来。
2. RSI 用样本并行和样本批内紧凑 plan，只为当前样本批真正用到的方向上传 plan。
3. 对 230k + S128 这类超大 plan，避免全量常驻，改为流式 sweep plan。
4. SI 的方向访问重复且有序，因此优先做设备端 plan 缓存。

5. RSI 的方向访问随机且样本批变化，因此优先做固定方向缓存、预热和并行加载。
6. RSI 的无环 sweep 在大网格上默认使用 fused wavefront，减少逐 level kernel launch，并保持和原 level-by-level wavefront 相同的依赖顺序。
7. RSI 与 RSI-tail 合并在同一批样本链中计算，避免重复随机链和重复 sweep。
8. mesh 数据按单元-面引用展平为 SoA，减少扫掠 kernel 中的间接访存。
9. 收敛范数、方向累加、样本 reduce 都在 GPU 上完成，只把最终场和极小控制量拷回 CPU。

整体上，SI 的优化重点是“少存完整角通量、少重复上传 plan、在无环依赖层内打开单元并行”；RSI 的优化重点是“样本批处理、只上传实际方向、复用方向 plan、把 RSI 和 tail 合并、减少大网格逐 level launch”。